

中国研究生操作系统开源创新大赛

项目说明书

作品名称:	StateBus
参赛队伍:	SynapseX
完成日期:	2026年7月

摘要

随着大语言模型应用由单一智能体任务处理逐步发展为多智能体分工协作，智能体之间需要持续交换任务描述、检索证据、计算结果、执行产物和历史经验。对于涉及国家秘密、商业机密和个人隐私的数据处理场景，为避免敏感数据在传输和远程调用过程中发生泄露，大模型及其智能体系统通常需要部署在本地可信环境中。然而，本地部署受到设备采购成本、运行功耗和计算资源等因素限制，往往难以运行参数规模较大的模型，只能采用参数量较小、单体能力相对有限的模型。如何通过多智能体协作弥补小模型在复杂任务规划、信息获取、工具执行和经验积累等方面的能力不足，成为本地智能体系统需要解决的重要问题。

现有多智能体系统通常使用自然语言或大段 JSON 数据传递中间状态，下游智能体需要重新解析任务上下文、证据内容和历史结果。这种通信方式虽然具有较强的通用性，但在多轮交互、长证据处理和连续任务执行过程中，容易产生上下文重复、额外词元开销和反复编解码等问题。同时，智能体执行过程中形成的语义状态、候选概率、计算结果和历史经验缺少统一的标识、兼容性约束与可审计复用机制，导致有限的本地算力被大量消耗在重复的信息组织、提示词预填充和结果解析过程中。针对上述问题，本文围绕本地有限算力环境下的多智能体高效协作，研究低开销通信、状态传递、共享记忆和受控工具执行方法，设计并实现了多智能体运行时系统 StateBus。主要内容概括如下：

1. 针对传统文本通信开销大、中间状态边界不清以及智能体之间依赖关系难以验证的问题，提出了一种基于类型化状态引用的多智能体通信机制。系统构建了由规划智能体、检索智能体、执行智能体和总结智能体组成的多智能体运行时，以 Unix 域套接字和 Protobuf 二进制消息协议作为正式控制通道，设计了语义状态引用、候选得分状态引用、执行产物引用和记忆对象引用等分层状态对象。不同智能体不再直接传递完整文本和大段结构化数据，而是通过状态引用访问经过标识和校验的中间结果，从而降低智能体之间的通信开销，并提高状态传递过程的可追踪性和可验证性。

2. 针对不同类型中间状态在生命周期、数据规模和复用方式上的差异，提出了一种分层状态存储与传递方法。短生命周期的语义嵌入向量、稠密数值状态和候选概率优先存放于共享内存，可回放对象及其清单存放于内存映射文件或内容寻址存储，程序执行结果存放于独立任务工作区，并通过执行产物引用传递。系统将闭集候选概率交由独立判定进程处理，使运行时能够根据候选得分执行受控重查和失败阻断。通过共同证据前

缀组织模型输入，触发同一 vLLM 实例的自动前缀缓存，减少重复的提示词预填充计算，降低本地模型在多智能体协作过程中的推理开销。

3. 针对多智能体连续任务中历史经验难以安全复用、不同任务之间状态兼容性难以保证的问题，提出了一种面向真实消费过程的共享记忆复用机制。该机制依次完成候选记忆召回、兼容性校验、角色重绑定、实际消费和复用效果记录，避免仅依据文本相似度直接注入历史内容。通过记录记忆对象的来源、适用角色、版本信息、兼容条件和实际效果，系统能够在保证任务边界和状态一致性的前提下复用已有证据、执行策略和计算结果，减少连续任务中的重复检索、重复推理和重复执行，增强本地小模型处理复杂任务的能力。

4. 针对模型生成程序存在越权访问、危险操作和结果不可控等问题，设计了受限 Python CodeAct 和结构化领域专用语言两种执行路径。模型生成的程序需要依次通过能力授权、抽象语法树审计、执行策略检查、bubblewrap 非特权隔离、输出契约检查和质量门验证，只有满足安全约束和结果要求的执行结果才能提升为可供后续智能体消费的正式产物。该方法在保留模型工具调用和程序生成能力的同时，限制其文件访问、系统调用、网络连接和资源使用范围，提高本地智能体执行过程的安全性、可控性和可审计性。

5. 为直观展示多智能体运行过程，设计并实现了 StateBus Studio 可视化交互系统。系统包括证据中心和实时演示界面。证据中心基于固定证据快照展示经过验证的实验结果，实时演示界面通过白名单任务配方、单工作进程队列和服务器推送事件流，展示任务在不同智能体之间的类型化流动、模型生成程序、产物验证过程和最终结论。系统前端采用 React、TypeScript、React Flow 和 ECharts 实现，后端采用 FastAPI 实现。

本文在统一、固定且可复现的代码和实验环境中开展了四层验证，包括 40 次同任务配置对照实验、20 次连续任务实验、10 个机制真实性验证用例和 25 个能力覆盖任务，共计完成 95 次正式执行，全部通过验证。与传统纯文本多智能体协作方案相比，完整 StateBus 方案将总词元数由 33,974 降低至 17,870，降低 47.40%；通信字节数由 36,069 降低至 12,677，降低 64.85%；总执行时间由 315.678 秒降低至 295.728 秒，降低 6.32%。消融实验结果表明，类型化结构通信使文本传递字节数降低 83.05%；共享记忆复用使任务执行时间减少 18.49%，词元开销降低 23.75%；CodeAct 工具使任务处理准确性提升 56%；本地推理前缀缓存机制在输入规模基本不变的情况下，将请求区间缓存块命中率由 0 提高至 78.02%，首词元时延和总延迟分别降低 88.3%和 54.6%。

综上，本文针对敏感数据本地处理场景下算力资源有限、小模型能力不足以及多智能体协作开销较大的问题，从类型化通信、分层状态传递、共享记忆复用、受控程序执行和前缀缓存优化等方面开展了研究。实验结果表明，StateBus 能够减少多智能体之间的重复文本传递和重复模型计算，提高执行结果的安全性、可验证性与可复用性，使参数规模较小的本地模型能够通过分工协作和状态共享，更加高效地完成复杂任务。

关键词：多智能体系统，本地大模型，状态传递，共享记忆，运行时优化，受控程序执行

目录

摘要	2
第 1 章 绪 论	7
1.1 目的与意义	7
1.2 项目背景	8
1.3 需求分解与设计目标	9
1.4 主要贡献	10
第 2 章 技术 理 论	11
2.1 设计思想	11
2.2 技术路线	12
2.3 代码原创说明	15
第 3 章 项目介绍	17
3.1 总体功能与实现架构	17
3.2 多智能体 协作与结构化运行时	19
3.3 非文本状态传递与引用化数据面	23
3.3.1 基于 SemanticStateRef 的语义状态传递机制	23
3.3.2 基于 LogitState 的受控重试机制	26
3.3.3 显式 KV 状态继承	27
3.4 共享记忆与跨任务复用	29
3.5 基于提示词布局的前缀复用机制	31
3.6 受限 Python CodeAct、结构化 DSL 与可信产物	33
3.7 软件界面、前后端与部署	35
第 4 章 实 验 测 试	39
4.1 测试环境说明	39
4.2 实验任务设置	40
4.3 结果总览	41
4.4 总体实验结果	42
4.5 结构化通信专项	43
4.6 非文本状态专项	44
4.6.1 StateRef 机制效果	44
4.6.2 LogitState 受控重试验证	45

4.6.3 显式 KV 状态继承实验.....	47
4.7 共享记忆专项	48
4.8 前缀复用专项	49
4.9 能力覆盖与工程稳定性	50
第 5 章 实现难点说明	51
5.1 控制指令与非文本状态的可靠传递	51
5.2 共享记忆的兼容校验与有效复用	52
5.3 模型生成代码的安全执行	53
5.4 多智能体运行的恢复与审计	54
5.5 Prefix Cache 的精确命中与可归因复用.....	56
5.6 显式 KV 状态继承的兼容绑定与生命周期治理.....	57
第 6 章 总 结	59
参 考 文 献	60
附 录	61
附录 A：主要交付文件	61
附录 B：实验指标口径说明.....	61
附录 C：正式实验任务与数据集说明.....	63
C.1 正式实验构成.....	63
C.2 两组连续任务.....	63
C.3 五类能力覆盖任务.....	65
C.4 数据来源与验证规则.....	65
C.5 显式 KV 状态继承实验任务	65
附录 D：StateBus Docker 与模型服务使用说明	错误!未定义书签。
D.1 Docker 配置与应用容器启动.....	错误!未定义书签。
D.2 外部 API 模型路由	错误!未定义书签。
D.3 本地 Qwen3-32B vLLM 模型服务.....	错误!未定义书签。
D.4 运行时模式与开关	错误!未定义书签。
D.5 显式 KV 状态继承服务	错误!未定义书签。

第 1 章 绪论

1.1 目的与意义

本项目围绕赛题一种面向多智能体协作的低开销通信、状态传递与共享记忆机制，构建面向多智能体协作的 StateBus 运行时。项目的目标不是简单串联多个大模型接口，而是针对协作过程中反复出现的通信冗余、中间状态传递、执行产物管理和历史经验复用等问题，形成可运行、可验证、可追溯的系统化解决方案。

随着智能体角色划分不断细化，系统需要交换的任务指令、证据、计算结果和历史信息也随之增加。若仍依赖完整自然语言历史或大 JSON 数据传递上下文，不仅会造成重复 Token 和额外通信开销，还会使对象身份、操作权限、结果状态和依赖关系隐含在文本之中。与此同时，历史经验的积极复用也可能将不兼容或已经失效的信息引入新任务。因此，多智能体系统不仅要降低通信开销，还需要保证消息合同稳定、中间状态真实消费、执行产物经过验证、历史记录满足兼容条件，并确保效率提升不会以任务质量下降为代价。

在通信层面，StateBus 将智能体之间的协作信息组织为动作、参数、能力、引用和回执等结构化字段。自然语言仍用于需要模型理解、分析和生成的环节，而任务控制、对象身份、权限范围和运行状态则由类型化协议表达。通过这种方式，上游智能体无需在每轮交互中重复传递相同证据、能力说明和历史上下文，下游智能体也无需从自然语言文本中反向解析步骤标识、操作参数和完成条件，从而降低模型上下文开销和通信载荷。

在状态层面，嵌入向量、候选得分矩阵和候选概率向量等中间对象保持原有数值形态，并分别通过 SemanticStateRef 或 LogitStateRef 在独立进程之间传递。系统记录数值对象的数据类型、形状、编码器或候选集合身份、来源清单、内容哈希和生命周期，使接收方能够校验其结构、来源和适用条件。SemanticStateRef 通过跨进程消费回执和证据选择变化验证语义状态的实际作用；LogitStateRef 由独立判定工作进程消费，并通过放行、受控重查或失败阻断证明数值状态能够改变运行时控制流。

在共享记忆层面，任务执行过程中形成的证据、策略、摘要和验证结果被组织为具有统一身份、来源信息和兼容条件的记忆对象^[1]。后续任务首先召回候选记忆，再依次检查任务意图、数据版本、输入输出结构、来源链和角色权限；只有通过兼容校验并被

目标智能体实际消费的对象，才被计入有效复用。该设计将检索到相似内容与发生有效复用区分开来，降低不兼容历史信息污染当前任务的风险。

1.2 项目背景

传统智能体协作通常由上游角色生成自然语言，下游角色再从自然语言中恢复动作意图、参数和中间事实。长文本虽然表达能力强，但缺少显式的结构约束、能力边界和拒绝语义。一个格式合法的 JSON 也不意味着其中的操作被授权、输入引用可访问、输出契约正确或结果可提交。

另一方面，检索增强生成通过检索外部知识并将相关文本注入生成上下文，以提升知识密集型任务表现^[2]。此类检索系统通常会生成语义向量与排序分数，但一些多智能体框架只把最终选中文本传给下游，数值中间态没有独立身份，也缺少跨进程消费证据。共享记忆也容易停留在检索到若干候选的统计层面：语义相似可能来自同一公司、同一指标或相似表述，却不代表统计口径、结构约束、任务意图和来源链兼容。

与完全依赖云端大模型接口相比，本地模型部署能够使任务数据、检索证据、执行产物和长期记忆保留在用户控制的运行环境中，减少敏感数据上传、外部服务依赖和网络传输带来的隐私与可用性风险。但单一本地模型参数规模、上下文容量和复杂任务处理能力通常受到计算资源限制，仅依靠一次模型调用难以稳定完成证据检索、数值计算、程序执行、结果验证和来源追踪等多阶段任务。因此，本项目不通过修改模型参数增强某个单模型，而是利用规划者、检索者、执行者和总结者的角色分工，将复杂任务拆分为可验证的协作步骤，并结合本地证据检索、非文本状态传递、受限代码执行和共享记忆复用，提高本地模型在复杂数据分析任务中的系统级处理能力^[3]。

本项目的语言模型服务和嵌入模型均在本地环境中运行。智能体之间通过 Unix 域套接字（Unix Domain Socket, UDS）传输控制消息，并以 Protocol Buffers（Protobuf）交换控制信息，大型证据、数值状态和执行结果保存在本地状态池、独立工作区或持久化存储中，浏览器端不能提交任意命令行指令，也不能启动或重启模型服务。步骤能力授权、输入引用白名单、非特权沙箱、验证器和记忆兼容门进一步限制每个角色可以访问的数据和执行的的操作，使数据不离开本地环境与本地模型不能任意访问全部数据同时成立。

1.3 需求分解与设计目标

表 1-1 赛题要求与项目落点

赛题要求	StateBus 对应实现	验证方式
不少于 3 个智能体	规划、检索、执行、总结 四角色	25 个能力任务；四角色链 路覆盖率 100%
结构化通信	UDS + Protobuf；动作、参 数、引用、能力、ACK 和 终态	50 条消息；control bytes 对 照
非文本状态	float32 语义状态、 StateRef、共享内存/mmap	9 个状态对象；跨 PID 消费 率 100%
共享记忆	MemoryRef、混合召回、兼 容门、消费与效果回执	20 轮连续任务；实际使用 为 35%
连续任务	Operating 与 Financial 两组 各 10 轮	20 轮任务；质量通过率 100%
CodeAct 加分项	受限 Python CodeAct、 AST/策略、bubblewrap 和 产物验证	18 个 CodeAct 任务；通过 率 100%，0 回退
openEuler 交付	单容器 Docker + openEuler 24.03 LTS-SP3	正式实验环境与交付环境 一致

系统首先追求低开销但不减少必要协作。完整链对照保持同样的角色关系和逻辑消息数，压缩的是消息载荷、重复证据和提示词的可见上下文，而不是通过删掉智能体或跳过质量门制造节省。与此相配套，引用化不能牺牲可验证性：控制面虽然只传类型化引用 Ref，但 Ref 必须解析到具有结构约束、哈希值、来源链、状态和生命周期的真实对象，任何裸路径或无法交叉验证的来源定位都不能成为正式输入。

系统其次遵循复用但不污染、生成但不直接信任。记忆检索与记忆复用被拆成两个阶段，未通过兼容门的候选不能进入角色输入；大语言模型可以产生计划和 Python 程序，但计划策略校验、步骤能力授权、AST 检查与策略审计、沙箱、验证器和提交门共同决定它们是否可执行和提交。所有阶段都记录结构化事件、引用哈希、验证状态和拒绝原因，使工程调试、实验统计和现场展示可以基于同一事实链。

1.4 主要贡献

项目以类型化对象管理任务的完整执行过程。任务首先被编译为 CanonicalTaskSpec，规划者据此生成候选计划，计划通过策略校验后形成 ApprovedPlan。检索阶段生成 EvidencePack，并根据任务需要发布 SemanticStateRef 或 LogitStateRef；执行阶段生成经过验证的 ExecutionArtifactRef；总结阶段依据证据和执行产物生成 ClaimSet。具有长期复用价值的内容经提交校验后，以 MemoryRef 写入共享记忆。各类对象通过任务、步骤、执行尝试、角色和内容哈希建立关联，使跨智能体传递过程能够被追踪和验证。Protobuf 与 UDS 控制面负责表达请求接收、执行启动、心跳保活、结果返回、失败处理、任务取消和资源回收等运行状态。

项目在数据面实现共享内存、内存映射文件、memfd 匿名内存文件和内容寻址存储（Content-Addressable Storage, CAS）等分层后端。SemanticStateRef 用于发布 float32 稠密语义状态，并记录跨进程消费、选择效果和释放回执；LogitStateRef 用于绑定候选集合、概率向量及生产者和消费者身份，并返回确定性判定结果；ExecutionArtifactRef 用于登记工作区输出、对象清单、验证状态以及从候选到已验证的完整生命周期。三类对象分别管理语义状态、候选概率状态和执行产物，避免使用模糊的统一状态引用掩盖验证方式的差异。

项目在知识与执行层实现了候选召回、兼容校验、角色视图重绑定、真实消费与行为效果记录的共享记忆闭环，同时提供受限 Python CodeAct 与 DSL 两条执行路径。模型生成程序必须经过能力校验、静态策略审计、轻量级进程沙箱工具 bubblewrap 的隔离、资源限制和结果验证；DSL 也必须满足结构化参数和输出合同，两种路径最终进入同一产物质量门。

项目还构建了覆盖完整链效率、连续任务、机制真实性与能力范围的分层实验体系，并以独立受控诊断验证前缀复用和 LogitState 受控重试，不将不同分母直接并入 95 次正式执行。实验结果固化为统一的 evidence snapshot。StateBus Studio 在此基础上提供固定证据页和真实任务页：前者保证正式数字不被临时运行覆盖，后者把四角色对象流、Python 程序与 DSL 指令、StateRef、MemoryRef、Artifact 和质量回执组合成可交互操作的产品界面。

第2章 技术理论

2.1 设计思想

StateBus 研究的对象不是某一个智能体的回答质量，而是多个智能体共同完成任务时，中间信息如何被表达、传递、验证和复用。传统多智能体系统通常把上一步得到的证据、参数和阶段结论重新写成自然语言，再作为下一步提示词的一部分。这样的方式便于快速搭建原型，却把几类性质不同的信息混在了一起：任务控制与业务证据使用同一文本载体，数值中间态需要先转成文本再由下游恢复，执行文件缺少独立的可信状态，历史经验也常被当成聊天记录直接注入新任务。随着角色、步骤和历史轮次增加，系统不仅会重复传输大量上下文，还很难判断下游究竟使用了哪一份状态、一次复用是否真的发生、失败产物是否已经被后续角色看到。

本项目采用合同驱动的设计思想，将多智能体协作理解为一组由运行时管理的对象状态变化。规划者、检索者、执行者和总结者仍然负责需要语言理解、检索判断、程序生成和结论组织的工作，但模型输出首先被视为候选：候选计划需要经过计划策略校验，候选代码需要经过能力授权和安全策略，候选产物需要经过业务验证器，候选记忆需要经过兼容判断。只有满足当前任务合同的对象才能被提升并交给下一角色。这样既保留了大模型面对开放任务时的适应能力，又把文件访问、能力范围、对象生命周期和可信状态收回到确定性的运行时中。

在系统分层上，StateBus 将控制面、数据面和记忆面分开。控制面表达由谁在什么条件下做什么，使用 Protobuf 描述动作、参数、角色、能力、引用和运行终态^[4]；数据面保存真正被处理的对象，包括证据、float32 语义状态以及执行产物，并通过类型化引用交接；记忆面保存跨任务可检索的摘要、策略和验证结果，同时保留来源、任务意图、输入输出合同与运行兼容条件。分层不是为了增加组件数量，而是为了让每一类信息采用与其生命周期相匹配的表示：小而频繁的控制消息直接在线路上传递，大对象留在状态池或工作区，长期对象进入可查询的持久存储。

这种设计还引入了生成权与批准权分离的原则。智能体可以提出步骤、证据候选、Python 程序和总结，但不能自行宣布这些结果已经可信；运行时根据任务哈希、能力授权、引用状态、内容哈希、验证器报告和来源链决定对象是否可以继续流动。一次执行进程正常退出，只能说明程序运行结束，不能直接说明业务答案正确；一条记忆与当前查询相似，只能说明它进入候选集合，不能说明它适合当前期间、结构约束或数据版

本；一段共享内存能够被打开，也不能替代数据类型、数据形状、编码器签名、对象清单和内容哈希的语义检查。

项目将失败路径看作设计的一部分，而不是正常流程之外的异常补丁。每个步骤同时具有稳定的步骤 ID，`step_id` 和独立的执行尝试 ID，`attempt_id`，重试不会覆盖前一次尝试；输入引用带有生命周期和消费条件，失败尝试生成的候选不能被总结者使用；工作进程失联、策略拒绝、验证失败和用户取消分别形成结构化终态，最后进入统一的资源结算与回收路径。通过这种处理，系统能够保留诊断证据，又不会让失败的中间对象沿智能体协作链继续传播。

上述思想最终落到三个相互约束的目标上：通信应当减少重复内容，但不能靠删减必要角色或质量门制造节省；状态应当保持原生数值形态跨进程传递，但必须具有身份、完整性和消费回执；记忆应当为后续任务减少重复工作，但必须先证明兼容并记录真实消费和行为效果。`StateBus` 不是一个把 JSON 换成 Protobuf 的序列化工具，也不是单独的共享内存或向量库封装，而是一套围绕对象合同、状态提升和证据闭环组织起来的多智能体运行机制。

上述设计可归纳为以下五项运行原则。

任务交接：以动作、能力、参数和引用组成稳定合同，避免下游从长文本中重新解析执行意图。

状态传递：控制面仅传递类型化引用，数值对象保留在分层数据面，并记录生产者、消费者和完整性回执。

产物可信：执行结果先作为候选产物保存，只有通过输出合同、业务验证和来源检查后才允许下游消费。

记忆复用：历史对象依次经过候选召回、兼容校验、角色重绑定、实际消费和效果记录，避免将相似召回等同于有效复用。

失败恢复：以稳定 `step` 和独立 `attempt` 隔离失败，保留已验证上游对象，并通过新授权和新工作区完成局部恢复。

2.2 技术路线

项目采用合同先行、机制分层、证据驱动的研究方法。首先从赛题要求反向建立可观测问题：结构化通信是否在消息数量不变时减少线路载荷，非文本状态是否由另一进程实际消费并影响选择，共享记忆是否通过兼容门进入后续任务，以及这些机制是否在

质量不下降的前提下形成端到端收益。随后为每个问题定义可执行合同、运行事件和验证分母，再实现智能体角色与物理载体，最后通过同任务对照、连续任务、机制专项和能力覆盖逐层验证。这样的顺序避免了先写一套演示流程、再从零散日志中寻找支持性数字。



图 2-1 StateBus 研究方法与技术路线

图 2-1 展示了从赛题问题到可交付系统的完整路线。任务进入系统后先被标准化为 CanonicalTaskSpec，明确任务意图、数据范围、允许操作、输出字段、质量条件和预算。规划者基于任务合同提出计划，运行时通过计划策略校验和能力目录将候选计划转为 ApprovedPlan；检索者在受限数据范围内形成带来源定位的 EvidencePack，并可将查询向量与候选向量作为非文本状态发布；执行者在一次性授权和独立工作区内运行受限 Python CodeAct 或 DSL，输出首先登记为候选产物；总结者只接收已经验证的证据与产物，并生成能够回溯来源的 ClaimSet。任务结束后，满足写回条件的对象以 MemoryRef 写入共享记忆，后续任务仍需经过检索和兼容校验才能使用。

支撑这条业务路线的是三层基础设施。UDS 与 Protobuf 构成控制面，负责请求、确认、开始、心跳、结果、取消和回收；SemanticStateRef、LogitStateRef、ExecutionArtifactRef、状态池、工作区和 CAS 构成数据与产物面，负责语义状态、候选概率、文件和验证状态；SQLite、FAISS、余弦检索及兼容策略构成记忆面，负责持久化、混合召回、过滤和回放。遥测模块与运行台账横向贯穿各层，把任务、步骤、执行尝试、角色、Ref、哈希值和质量结果关联到同一条调用轨迹中。产品界面和实验汇总器不另造数据，而是从这些结构化对象和事件中投影出流程、指标与证据。

实现顺序也遵循这一依赖关系。项目先建立独立实现包、正式任务注册表和合同模型，再完成 Protobuf/UDS 与工作进程状态语义；在任务和控制合同稳定后接入 EvidencePack、SemanticStateRef、LogitStateRef、分层存储、工作区和产物生命周期；记忆复用建立在可验证输入、产物和运行签名之上；随后增加 CodeAct 隔离、前缀布局、

统一遥测、正式基准测试和 StateBus Studio。越靠后的能力只能消费前面已经闭合的对象，而不能绕过合同直接访问文件或把临时结果写入长期记忆。

当前前缀复用通过共同证据布局、兼容身份和依赖安全调度，触发同一本地大模型推理引擎 vLLM 实例的 APC；vLLM 中的键值缓存块（Key-Value Cache Block，KV Block）始终由推理引擎创建、保存和淘汰。正式 95 次基线中的非文本状态仍聚焦 SemanticStateRef，LogitState 受控重试和前缀复用作为补充机制诊断单独报告。部署采用 Docker 与 openEuler 24.03 LTS-SP3，以满足交付复现；Docker 负责外层环境一致性，bubblewrap 负责生成代码的内层最小执行边界，两者不混写为同一种安全机制。

表 2-1 技术路线的阶段、合同与验证方式

阶段	稳定对象或机制	解决的问题	主要验证方式
任务标准化与计划批准	CanonicalTaskSpec、ApprovedPlan、CapabilityGrant	将自然语言目标转化为可检查的执行边界	检查 schema、依赖图、预算、角色和能力
结构化控制	Protobuf 消息、ACK、心跳和终态事件	减少控制冗余并显式表达运行状态	对比消息数、control bytes 和 wire bytes
非文本状态与产物	SemanticStateRef、LogitStateRef、ArtifactRef、清单和回执	保持数值形态并隔离候选产物	检查跨 PID 消费、哈希、选择/授权效果和验证器
共享记忆	MemoryRef、混合召回和兼容门	在不污染当前任务的前提下复用历史	统计实际使用、效果、拒绝原因和跳过步骤
可信执行	受限 Python CodeAct、结构化 DSL、工作区和提交门	限制生成程序的权限与输出范围	检查策略、隔离环境和业务复算
统一证据与产品化	遥测、证据快照和 Studio	让实验、本文和系统界面使用同一事实链	核对固定基线、正式运行和快照一致性

2.3 代码原创说明

StateBus 的任务合同、计划策略、Protobuf 消息语义、运行时状态机、类型化引用、分层状态路由、语义状态编码与消费、证据定位、记忆兼容门、CodeAct 策略、工作区与产物生命周期、遥测、基准测试汇总及 Studio 业务逻辑均在本项目中实现。其中，控制协议和模式定义文件中的消息描述由项目定义，运行时使用 Google Protocol Buffers 公开 API 构造消息类；项目没有把第三方协议实现复制为自己的控制面。

仓库中的 third_party/仅保存开源项目调研与对照材料，pytest 配置将其排除在测试递归之外，主线和 Studio 也没有从该目录导入运行模块。这些代码不参与正式打包、智能体执行或本文实验。规划者、检索者、总结者及自适应 CodeAct 通过兼容 OpenAI API 的 HTTP 接口调用既有 vLLM，项目不包含也不改写模型源码；提示词合同、输出解析、重试、Token 与时延采集和健康复用由 StateBus 实现。

Studio 的 FastAPI/React 组件通过公开包接口调用，Docker 与 openEuler 用于交付环境复现，不作为项目原创算法。pytest、Matplotlib 和 Pillow 分别用于测试与文档制图，不进入运行时的核心协作协议。本文所述系统行为以当前项目实现为准，不将上游源码改名后并入项目主实现。

表 2-2 第三方组件引用及项目实现边界

第三方组件	使用位置及用途	项目自行实现的部分
Google Protocol Buffers	/control/: 使用公开 API 完成消息编码与解析	消息 schema、Ref 字段及 ACK、心跳、终态和回收语义
Python 标准库 (UDS、共享内存、mmap、SQLite)	/control/、/state/、/memory/: 进程通信、状态载体和索引	长度帧、载体路由、对象注册、租约、回执和记忆事务
NumPy	/state/: float32 编码、校验和数值计算	行级合同、HydrateManifest、编码器签名和跨 PID 回执
FAISS / sentence-transformers	/memory/: 向量编码与候选检索	多路召回、RRF 融合、兼容门、角色视图和效果记录
bubblewrap	/runtime/: 通过 bwrap 命令提供进程隔离	就绪检查、能力授权、AST/路径策略、资源限制和提交门

第三方组件	使用位置及用途	项目自行实现的部分
FastAPI / React	/studio/、studio-ui/：API、SSE 和可视化界面	配方白名单、单工作进程队列、Run 恢复及证据/实时页面

开源组件的著作权和许可证归原项目所有，部署时按依赖清单安装。项目贡献的是这些基础能力之上的对象模型、运行语义、验证闭环和可复现实验组织。

第 3 章 项目介绍

3.1 总体功能与实现架构

StateBus 提供一条从真实任务输入到可验证结论的完整多智能体链。用户可以选择离线财务报告或运营指标任务，系统随后完成任务标准化、计划、检索、非文本语义选择、受控执行、结果验证、总结和可选记忆写回。项目并未把协议、状态池、记忆库和 CodeAct 做成彼此独立的展示脚本，而是让它们共同服务于同一个任务合同：控制面负责推进步骤，数据面负责保存证据和数值对象，产物面负责区分候选文件与可信结果，记忆面负责跨任务召回与安全复用。

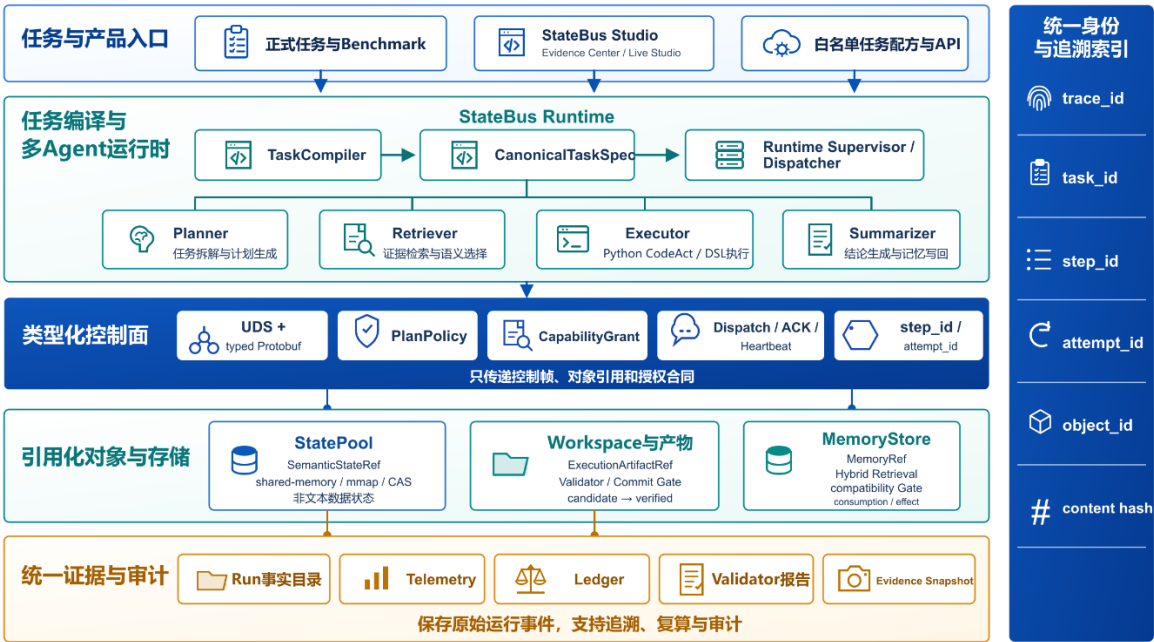


图 3-1 StateBus 系统总体架构图

图 3-1 中，入口层由基准测试、任务目录和 StateBus Studio 构成。任务编译模块从注册任务、数据清单和用户允许参数生成 CanonicalTaskSpec，随后由运行时管理规划者、检索者、执行者与总结者。UDS/Protobuf 只传递控制帧和 Ref，状态池、任务工作区与 CAS 保存真实对象，记忆存储模块保存经过提交门的长期记忆，遥测与运行台账记录所有关键交接。各层之间没有共享可变的全局 Python 对象，下游只能按合同解析已登记的 Ref；因此同一条链既可以被正式基准测试调用，也可以被 Studio 以事件方式还原。

一次任务运行的建立从任务身份开始，而不是从调用某个智能体开始。任务目录把数据集标识、数据清单、任务类别、操作类型、目标实体、时间范围、必需输出、允许

工具和资源预算组合为编译输入；正式模式下，任务编译模块只接受注册过的任务类别、任务意图、输出字段和工具名称，并把复合字段、排序后的参数与结构版本纳入规范载荷。由此生成的任务规范哈希会继续进入计划、检索、授权、产物和记忆对象。后续如果数据范围、输出合同或工具集合发生变化，即使自然语言问题相似，也会形成不同的任务身份，避免两个表面相近而实际口径不同的任务共享同一条执行链。

运行时内部不是一个把四个 Python 函数依次调用完的循环。运行时驱动模块持有 Run 级调用轨迹、任务合同、批准计划、对象注册表和事件发射器；执行监督模块管理每个步骤的执行尝试状态；能力分发模块负责将批准步骤投影成目标角色真正可见的请求；状态存储模块、产物生命周期管理模块与记忆索引模块分别管理短期数值对象、执行产物和长期知识。组件间通过不可变数据类、哈希值和显式状态迁移衔接，使业务对象的生命周期与进程生命周期解耦：工作进程退出并不代表产物可信，文件存在也不代表它可以被下一角色消费。

表 3-1 主实现目录与职责

路径	主要职责
/contracts/	定义任务合同、运行兼容签名和执行表示合同
/control/	实现类型化消息、UDS 通信和工作进程控制
/refs/、/state/	管理类型化引用、对象注册和分层状态存储
/retrieval/、/provenance/	完成受限检索、证据定位和来源追踪
/memory/	负责记忆持久化、召回、兼容判断和提交
/runtime/	负责编译、调度、策略、安全执行、回放和遥测
/benchmark/	管理正式任务、对照实验、评分和报告
/studio/、studio-ui/	提供运行服务、白名单作业和产品界面
tests/、docker/、scripts/	提供回归测试、容器环境和交付脚本

系统对外呈现两类能力。Evidence Center 面向实验审阅，读取固定证据快照并展示质量、Token、线路传输字节、时延、状态和记忆指标；Live Studio 面向任务运行，允许用户从白名单选择真实任务，并查看任务在四个智能体之间如何转化。两类页面共用相同的运行时和数据模型，但固定证据不会被临时运行自动覆盖，避免即时模型波动改变正式实验结论。

3.2 多智能体 协作与结构化运行时

StateBus 运行时是四类智能体协作的控制中心，负责将外部任务转换为具有明确角色、依赖关系、输入引用和输出合同的执行图，并管理任务从计划生成、能力授权、步骤分发到结果验证的完整过程。规划者、检索者、执行者和总结者仍然承担语言理解、证据检索、程序执行和结论组织等工作，但各智能体不能自行扩大权限、访问任意对象或直接宣布结果可信；所有角色之间的交接都必须通过运行时登记的任务合同和类型化引用完成。

任务进入系统后，任务编译模块将输入整理为统一的任务合同，并以 CanonicalTaskSpec 对象保存。该对象描述当前任务的目标、允许使用的能力、需要访问的输入对象、预期产物及完成条件，为后续角色提供稳定的协作边界。运行时不会直接将原始自然语言请求和完整历史交给所有角色，而是根据当前角色和执行阶段，从 CanonicalTaskSpec 中投影出该角色真正需要的任务信息。

任务编译模块同时区分正式基准任务与交互任务。正式基准任务必须使用已经登记并经过检查的 CanonicalTaskSpec，以保证不同实验运行中的任务范围、输入对象、输出合同和统计分母保持一致；交互任务可以从结构化输入或自然语言请求生成任务合同，但无法可靠结构化的内容只能作为自由交互任务运行，不能进入正式实验基线。两种入口最终使用相同的运行时执行链，区别仅在于任务合同的生成方式和可信等级。

规划者接收 CanonicalTaskSpec、能力目录以及当前可见的输入引用，并生成 PlanProposal，即尚未获得执行权限的候选计划。计划中只描述步骤、执行角色、依赖关系、所需能力和预期输出，不直接授予执行权限。语义计划解析模块将模型输出转换为结构化计划，计划策略校验模块再从任务一致性、依赖图合法性、角色与能力匹配、资源预算和输出合同等方面进行检查。

只有通过策略校验的 PlanProposal 才能提升为 ApprovedPlan。对于仅存在格式问题的计划，系统允许进行有限的结构修复，但修复不得改变任务目标、步骤依赖、能力范

围、资源预算或失败策略；如果候选计划仍然无法通过检查，运行时只能使用已经登记的备用计划或结束当前任务，不能接受模型通过重新表述获得额外权限。

ApprovedPlan 形成后，运行时按照计划中的依赖关系创建步骤状态，并为满足执行条件的步骤签发一次性 CapabilityGrant。该授权对象将当前任务、步骤、执行尝试、执行角色、批准能力、允许访问的输入引用、输出合同、工作区和有效时间绑定在一起，使一次授权只能用于指定步骤和指定执行尝试。

执行监督模块负责管理步骤状态、执行尝试、接收确认、心跳、超时、取消和资源结算；自适应能力分发模块负责在真正发送请求前，再次核对 ApprovedPlan、CapabilityGrant、目标角色和当前输入对象。只有计划、授权和实际请求保持一致，任务才会被分发给对应工作进程。

在组装角色请求时，运行时从引用注册表中解析当前步骤获准访问的 EvidencePack、SemanticStateRef、ExecutionArtifactRef 和 MemoryRef，并根据目标角色生成最小可见输入。模型不能通过输出内容增加未经授权的引用，前端也不能利用额外参数扩大能力范围。由此，智能体之间传递的是经过运行时投影和验证的任务对象，而不是不断累积的完整自然语言历史。

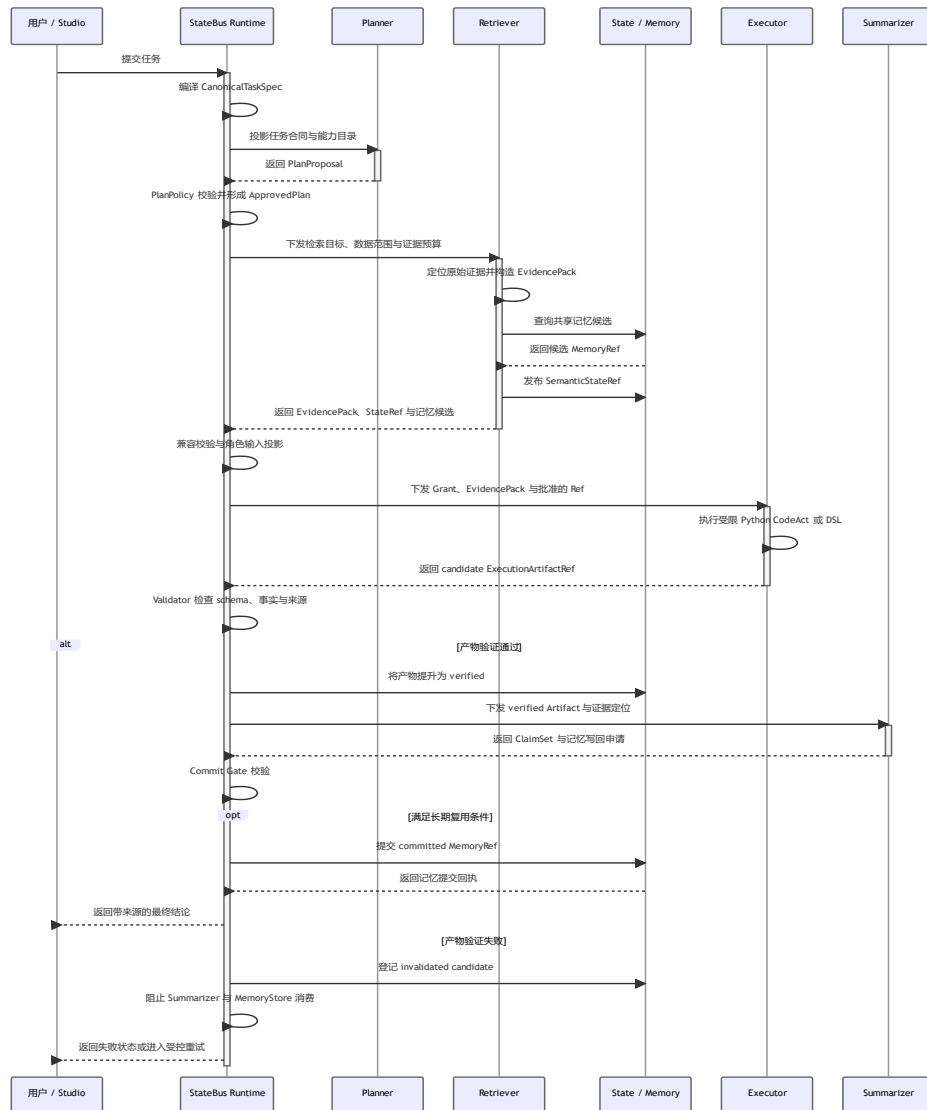


图 3-2 单任务协作时序图

如图 3-2 所示，ApprovedPlan 形成后，运行时按照依赖关系依次调度规划者、检索者、执行者和总结者。每个角色只接收当前步骤所需的任务信息、批准能力和输入引用，不共享完整对话历史。上游角色的输出也不会直接拼接到下游提示词中，而是先被登记为具有明确类型、身份和状态的中间对象，再由运行时根据目标角色生成最小可见输入。

检索者根据 ApprovedPlan 投影出的检索目标、数据源范围和证据预算定位原始材料，并将选中的文本片段、表格位置、来源哈希和定位信息封装为 EvidencePack。该对象用于保存可验证的证据内容及其来源。需要比较多个语义候选时，检索者还可以发布 SemanticStateRef，使候选向量和得分矩阵保持数值形态，并由后续角色所在的独立进程实际消费。

共享记忆只作为检索者阶段的候选来源之一。记忆存储模块返回的历史对象首先进入候选集合，运行时随后检查其任务意图、输入输出结构、数据来源、运行条件和角色可见性。只有通过兼容校验的 `MemoryRef` 才能被投影到目标角色；仅被检索到但未通过校验的历史内容不会进入角色输入，也不会被统计为有效复用。

执行者接收经过验证的 `EvidencePack`、获准访问的 `StateRef` 或 `MemoryRef`、当前步骤的 `CapabilityGrant` 以及独立工作区。根据任务操作和授权范围，执行者可以生成受限 `Python CodeAct` 程序，也可以生成结构化 DSL 指令。两种执行路径都只能访问 `CapabilityGrant` 所登记的输入对象，并按照预先声明的输出合同生成结构化结果。

执行者产生的文件或 JSON 结果首先被登记为候选 `ExecutionArtifactRef`。该对象用于保存执行输出、内容哈希、来源关系和当前验证状态。进程正常结束只表示程序完成运行，不能直接证明业务结果正确；验证器还需要检查输出结构、字段完整性、数值类型、计算参数、内容哈希、来源关系以及任务规定的确定性事实，必要时使用独立程序重新计算关键结果。

只有通过全部检查的候选产物才能被提升为已验证的 `ExecutionArtifactRef`，并获得下游可见性。验证失败的产物仍保留在当前执行尝试的工作区中，用于错误分析和审计，但不能被总结者或记忆存储模块消费。这一机制将程序执行成功和结果可以进入正式业务链区分开来。

总结者只接收已经验证的执行产物及其原始证据定位，并生成 `ClaimSet`。该对象用于组织最终结论及其引用关系，每条结论都需要关联对应的 `EvidencePack` 或 `ExecutionArtifactRef`，使最终答案能够回溯到具体输入、计算结果和验证报告。若摘要、分析策略或执行产物满足长期复用条件，总结者可以提出记忆写回申请；申请通过提交校验后，相关内容才会以 `MemoryRef` 写入共享记忆。

表 3-2 四角色输入、输出和交接条件

角色	主要输入	主要输出	交接前检查
规划者	<code>CanonicalTaskSpec</code> 、能力目录、预算和允许的历史视图	<code>PlanProposal</code>	依赖图、角色、 <code>capability</code> 、预算和 <code>plan hash</code>

角色	主要输入	主要输出	交接前检查
检索者	ApprovedStep、来源范围和证据预算	EvidencePack、SemanticStateRef	来源定位、source hash、manifest 和 encoder contract
执行者	CapabilityGrant、已验证输入 Ref 和独立工作区	Python 程序/DSL 指令、候选 ArtifactRef	策略、沙箱、输出 schema 和业务验证器
总结者	EvidencePack、已验证 ArtifactRef 和兼容 MemoryRef	ClaimSet、MemoryCommitProposal	引用、来源链、质量门和写回条件

3.3 非文本状态传递与引用化数据面

3.3.1 基于 SemanticStateRef 的语义状态传递机制

非文本状态传递是 StateBus 区别于纯文本协作的核心实现之一。这里的非文本并不是把向量转成 Base64 编码或 JSON 数组后继续塞入消息，而是让查询嵌入向量、候选嵌入向量和稠密选择矩阵保持数值形态，采用小端字节序的 float32 格式，并按行优先顺序连续存储。真实字节存放在共享内存或内存映射文件中，控制面只携带

SemanticStateRef。消费者在独立进程中解析同一物理对象，完成语义排序或证据选择，再通过结构化结果返回候选标识、行号、得分、生产者进程 ID、消费者进程 ID 和编码器签名。

发布前，运行时检查向量维度、有限值和归一化状态，并为矩阵生成解析清单。该清单将行号与候选标识、证据来源、业务类别和字节提示绑定，因此下游返回行号后，运行时能够确定它选择了哪条真实证据，而不是只获得一个脱离来源的分数。矩阵、解析清单和来源文档哈希共同构成状态身份。

发布过程由数值数据块与伴随元数据同步完成。检索者将查询向量与候选向量按行堆叠，转换为连续的小端 float32 字节，并计算数据块哈希；同时生成包含数据形状、数据类型、行级合同、编码器签名、候选来源定位和对象清单哈希的伴随元数据。状态存储模块先做存储决策，再创建物理对象，状态注册表最后登记逻辑 Ref 和租约。只有物理对象、伴随元数据与状态注册表三者均成功，运行时才发出状态已发布事件。任一阶段失败都会回收已经创建的部分对象，避免控制面出现无法解析的悬空 Ref。

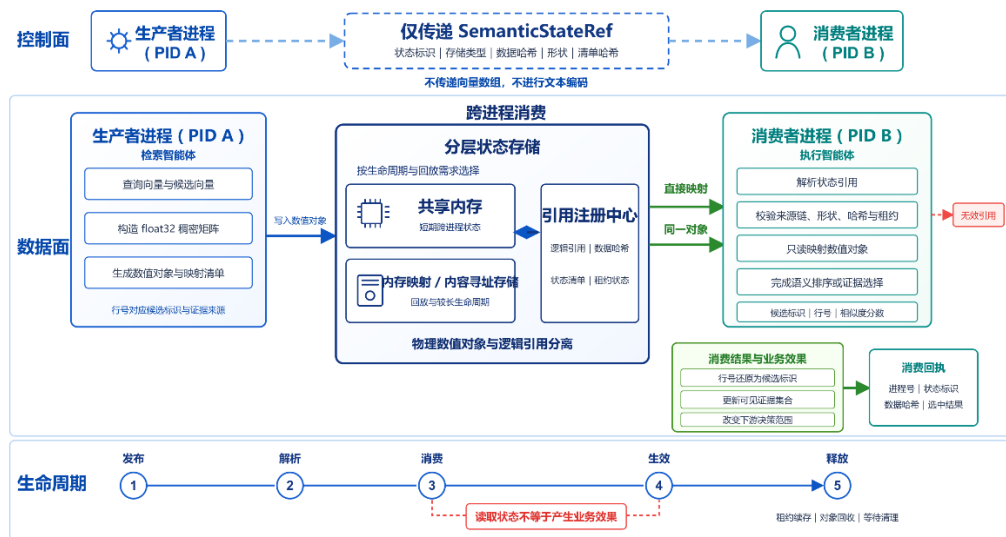


图 3-3 StateRef 非文本状态生命周期

分层状态存储模块根据对象类型、大小、生命周期和消费者可解析能力选择载体。短期稠密语义状态优先进入共享内存，以便不同进程 ID 直接映射；需要回放或保存更久的状态可进入内存映射文件或 CAS；执行结果不复用这类数值状态合同，而是进入任务工作区并形成 ExecutionArtifactRef。发布时，状态注册表记录状态标识、存储类型、数据块哈希、对象清单、结构约束与元数据，控制面只看到逻辑句柄。若共享内存不可用，系统可以按明确策略选择可解析的内存映射后端，并把回退原因写入存储决策记录，而不是静默改变语义。

分层存储策略不单纯按大小选择载体，而是同时查看对象类型、预期生命周期、回放要求、跨进程消费者和后端能力：短时稠密矩阵优先使用共享内存，需要跨运行回放或长期保存的对象使用内存映射文件或 CAS，EvidencePack 和对象清单采用可校验的结构化文件，执行输出始终留在当前执行尝试的工作区，之后再提升为 ArtifactRef。每次选择都会记录请求后端、实际后端和原因；如果消费者不具备某种载体的解析能力，系统会在发布前选择明确的兼容路径，而不是让消费者在运行中猜测字节含义。

消费端取得 Ref 后不会立即读取。它先比对任务与步骤来源链、状态类型、长度、数据块哈希、数据类型、数据形状、编码器签名、对象清单哈希、状态是否活跃以及租约是否有效，再以只读方式映射数值对象。计算完成后，成功结果中的消费回执与 Ref 注册表交叉核对；只有生产者与消费者、对象哈希和选择结果闭合，运行时才把选中标识应用到 EvidencePack 的可见集合。这意味着 StateRef 的作用发生在 LLM 提示词构造之前，能够直接改变下游候选决策集合。

在消费者进程中，解析器依据伴随元数据构造只读数值数组，并再次检查总字节数是否与行数×维度×4 一致。相似度计算返回行号后，HydrateManifest 将其反向映射为候选标识和来源定位；运行时对照请求前后的候选决策集合哈希，记录候选集合是否真的改变。回执中的生产者进程 ID、消费者进程 ID、状态标识、数据块哈希、选中标识和得分必须与状态注册表闭合。这样既能证明字节跨越了进程边界，也能证明这些字节参与了业务选择，而不是被读取后丢弃。

对象结束使用后，运行时减少租约，进入释放状态并删除共享内存或回收内存映射资源；重复释放按幂等操作处理。遥测分别记录 publish、resolve、consume、效果和 release，避免把对象创建了误写为对象使用了。实验中的 9/9 跨 PID 消费、9/9 改变选择和发布/释放字节闭合，正是从这些独立事件与回执中核验，而不是从编码器调用次数推测。

状态清理同样由运行时驱动。正常完成、消费校验失败、工作进程 trap、取消和重试都会进入统一资源结算；状态注册表先阻止新的 resolve，再等待或撤销租约，随后关闭共享内存句柄、unlink 所有者对象或回收内存映射文件。释放事件保存对象大小与终态，重复 GC 只返回幂等结果。此设计解决了连续运行中常见的后续运行污染问题：上一 Run 即使失败，也不能把活跃 Ref、旧 PID 或未关闭映射带入下一 Run。

StateBus 特意将 SemanticStateRef 与 ExecutionArtifactRef 分开。前者面向短期数值状态，关注数据类型、数据形状、编码器、载体和消费；后者面向两类执行路径的输出，关注工作区、文件哈希、输出结构、验证器和候选/已验证生命周期。MemoryRef 则面向跨任务对象，关注来源角色、主题、兼容签名和提交状态。三种引用都包含身份与来源，但不能用同一套读取和验证规则互相替代。

表 3-3 类型化引用与物理载体

引用类型	典型内容	主要载体	消费前关键检查	结束方式
SemanticStateRef	embedding、query/候选 float32 矩阵	优先共享内存，可选内存映射文件或 CAS	dtype、shape、encoder、对象清单、hash 和 lease	消费回执后释放并回收

引用类型	典型内容	主要载体	消费前关键检查	结束方式
LogitStateRef	候选概率向量和 other_mass	共享内存	候选面、hash、shape、lease 和进程身份	判定门回执后释放并记录 tombstone
ExecutionArtifactRef	JSON、表格、计算结果和报告	任务工作区 + artifact root/CAS	输出 schema、blob hash、验证器和验证状态	验证通过后可见；失败则失效
MemoryRef	摘要、策略、证据链和已验证产物视图	SQLite + embedding/index + 伴随元数据 /CAS	intent、结构约束、对象清单、lineage 和运行签名	提交、辅助/复用或拒绝

3.3.2 基于 LogitState 的受控重试机制

SemanticStateRef 主要用于传递 Embedding 和候选选择矩阵，而在执行者进行闭集能力选择时，运行时还需要判断模型当前选择是否具有足够的数值依据。普通文本路径只能得到模型最终选择的候选，例如选择 detect_outliers，无法区分模型对该选择高度确定，还是多个候选概率接近后偶然输出了其中一个。由模型额外生成文本置信度仍属于模型自述，不能直接作为执行授权依据。

为此，StateBus 在执行者候选选择与 CodeAct、结构化 DSL 或其他业务工作进程真正执行之间增加了基于 LogitState 的受控重试门。执行者首先在 2~8 个已批准候选组成的闭集中返回唯一候选编码。系统从该选择 Token 对应的候选对数概率中提取各候选的真实概率，并保留候选集合之外的剩余概率质量。虽然对象名称为 LogitState，其当前载荷实际保存的是候选概率向量，而不是模型归一化前的原始 Logit 张量。

候选概率按照固定候选顺序编码为小端 float32 数据，通过共享内存发布，并以 LogitStateRef 交给独立判定工作进程。UDS 与 Protobuf 控制面只传递状态引用、候选集合身份和运行信息，不在消息中内联概率数组。判定工作进程在读取前检查状态标识、候选集合摘要、载荷哈希、数据形状、租约以及生产者和消费者身份，避免将某一次候选选择产生的概率状态用于其他候选集合或其他执行尝试。

当模型实际选择等于最高概率候选且概率差值不低于 0.10 时，判定进程返回接受；否则返回重试。若首次判断为 **RETRY**，单次重试模式（**retry_once**）允许执行者在同一候选闭集中重新检查一次；第二次仍不满足条件、状态不可用或回执校验失败时，运行时执行失败关闭，不启动下游 **CodeAct**、**DSL** 或业务工作进程。系统默认关闭该机制，仅观测模式只记录判定结果而不改变控制流，**retry_once** 模式才会执行受控重查和失败阻断。

状态无论被接受、拒绝还是在处理中发生异常，都会通过统一的 **finally** 路径释放共享内存和元数据，并留下释放记录。由此形成真实候选概率—非文本状态发布—独立进程消费—确定性判定门回执—运行时控制流变化—状态释放的完整闭环。

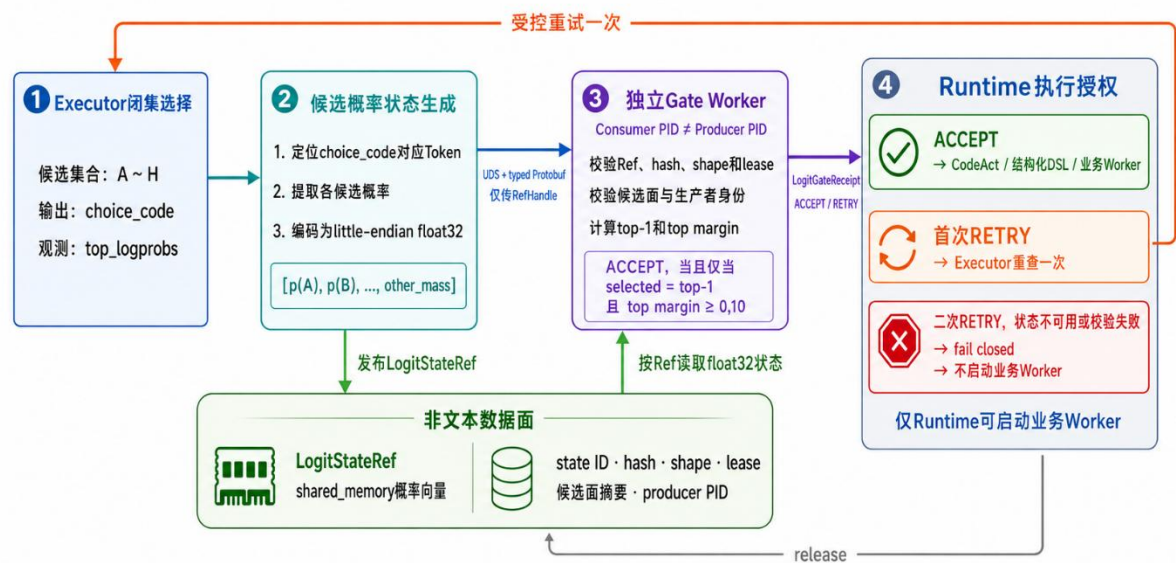


图 3-4 LogitState 受控重试与执行授权机制

如图 3-4 所示，LogitState 将模型闭集选择位置的候选概率转换为可验证的非文本对象，并由独立进程执行确定性判定门规则；运行时根据结构化回执决定放行、重查或阻断业务执行。

3.3.3 显式 KV 状态继承

在多智能体主链中，相邻角色可能需要读取相同的长证据。即使系统不再传递完整对话，下游模型仍可能重复执行相同证据的预填充计算。为减少这部分神经计算与请求传输，StateBus 在执行者与总结者之间增加了显式 KV 状态继承机制。

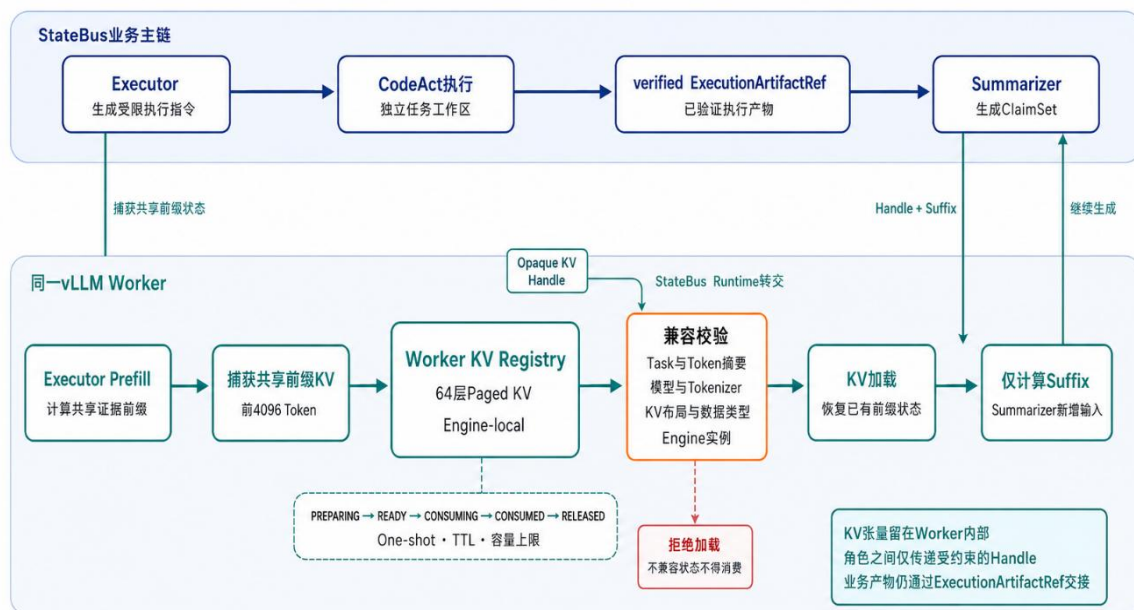


图 3.5 显式 KV 状态继承机制

如图 3-5 所示，系统同时保留业务主链和 KV 状态旁路。业务主链中，执行者根据获准的任务步骤生成受限执行指令，CodeAct 在独立工作区中完成计算，执行结果经过验证器检查后登记为已验证的 ExecutionArtifactRef。总结者仍然通过该产物引用获取业务结果，因此 KV 机制不会替代原有的执行、验证和产物交接过程。

在 KV 状态旁路中，执行者调用模型时首先对共享证据前缀执行预填充，推理引擎从本次计算中捕获指定前缀对应的多层分页式 KV 缓存，并将其保存在工作进程内部的短生命周期 KV 状态注册表中。运行时得到的只是不透明 KV 状态句柄，该句柄用于定位状态，不包含完整 KV 张量，也不直接携带原始证据文本。

总结者开始生成前，运行时将 KV 状态句柄和总结者新增的后缀输入提交给同一推理引擎。系统首先检查任务身份、前缀 Token 摘要、模型和分词器版本、KV 布局、数据类型及引擎实例。只有全部条件一致，工作进程才会加载已有 KV 状态，使总结者继承共享前缀的计算结果，并只对新增后缀执行前向计算；校验失败的状态不得进入消费阶段。

KV 状态按照准备中、就绪、消费中、已消费和已释放的生命周期流转。每个 KV 状态句柄采用一次性消费，并受到生存时间（Time to Live, TTL）、容量上限和异常失效机制约束。总结者完成生成或发生异常后，运行时都会释放对应状态，避免同一 KV 状态句柄被重复使用或长期占用显存。

嵌入矩阵具有稳定的数据形状和数据类型，可以通过共享内存或内存映射文件由不同进程读取；KV 状态则依赖具体模型结构、分词器、并行配置和工作进程内部布局。

该机制能否在计入 KV 捕获、加载和释放成本后获得端到端收益，将在第 4 章的显式 KV 状态继承实验中进一步验证。

3.4 共享记忆与跨任务复用

共享记忆的目标不是提高检索到历史的比例，而是在连续任务中让经过验证的历史对象安全地减少重复工作。记忆索引模块保存 **MemoryRef**、结构化嵌入向量和提交状态；持久模式下使用轻量级数据库 **SQLite** 保存索引与元数据^[5]，向量候选可由稠密向量索引库 **FAISS** 的归一化内积检索产生^[6]，**FAISS** 不可用时使用受控余弦计算。关键词、标签和向量三路候选经过倒数排名融合（**Reciprocal Rank Fusion, RRF**）合并，这一步只决定候选顺序，不决定是否允许复用。

混合查询由 **MemoryQuery** 对象驱动，该对象携带当前任务及任务规范哈希、查询文本、标签、查询嵌入向量、允许的记忆类型、目标角色和数量上限。记忆索引模块分别取得关键词、标签与向量检索的前 K 项，记录每路排名，再使用固定参数完成 **RRF**；相同得分按稳定的记忆标识排序，保证重复运行的候选顺序可解释。**SQLite** 全文检索承担持久元数据和关键词查询，**FAISS** 使用归一化向量的内积近似余弦；缺少 **FAISS** 时回到逐项余弦，并保留命中来源信息。召回阶段输出完整候选池、分类信息和重排记录，尚未把任何摘要注入角色输入。

每条 **MemoryRef** 除记忆 ID、来源角色、创建时间、主题和摘要外，还记录记忆类型、任务及任务规范哈希、输入输出结构约束、数据对象清单、来源链、验证状态、运行兼容签名和回放类别。后续查询同样携带当前任务意图、允许的记忆类型、目标角色、数据范围和运行签名。兼容门逐项比较这些字段；语义分数再高，只要期间、单位、结构约束、数据来源或运行环境不满足当前合同，就只能记录不兼容原因，不能进入角色输入。

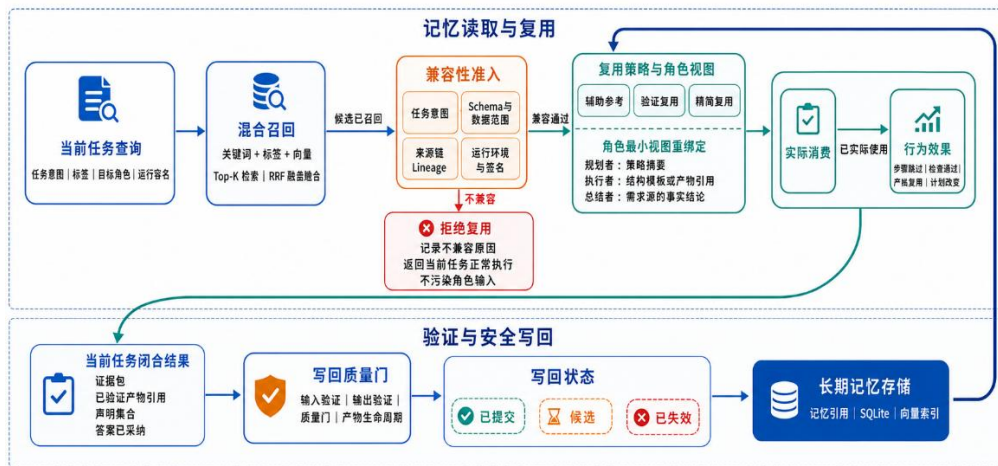


图 3-6 共享记忆任务复用和写回闭环

通过兼容门的记忆还要重绑定为当前角色可以看到的视图。规划者只获得可用于计划的策略摘要，执行者获得已验证的操作模板或产物引用，总结者获得带来源的事实和结论；一个角色不能因为持有记忆标识就读取整条历史对象。运行时随后记录 MemoryRef 是否真正进入角色输入、是否改变了计划、证据或执行路径，以及是否跳过步骤或模型调用。项目将查询分为发现候选、通过兼容校验、实际消费和产生行为效果四层，只有实际消费才计入实际使用，只有可观察的步骤变化才计入效果。

角色重绑定由运行时生成新的最小视图，而不是把数据库行原样复制进提示词。视图保存 source memory ID、目标角色、允许字段和视图哈希；真正构造角色输入后，系统再写入消费 record，记录 consumed-by、进入了哪个 step、采用 assist/replay 哪一类策略以及输入候选决策集合。步骤跳过、检索范围缩减、采用历史产物或执行计划变化分别形成效果字段。只有候选但未生成角色视图，或生成视图后未进入实际请求，都不会计入实际使用，从统计层面阻止召回即复用的口径膨胀。

复用强度也有边界。assist 类记忆只能提供提示，不能跳过当前计算；validated replay 要求历史产物能够在当前输出合同和验证器下重新确认；精确匹配 replay 还要求任务、输入对象清单和运行兼容签名更严格一致。任何条件不满足时，系统回到当前任务的正常检索和执行路径。拒绝历史不是错误，而是一种安全结果，它说明系统没有用高命中率掩盖不兼容数据。

新记忆只能从已经闭合的来源链写回。总结者或运行时提出记忆写回申请后，记忆存储模块检查上游 EvidencePack、已验证 ArtifactRef、ClaimSet、质量门和答案已采用状

态，再决定保存为已提交、候选或已失效。失败执行尝试的文件即使仍在工作区，也不会成为长期知识。这样的写回约束与读取时的兼容门共同防止错误历史在连续任务中累积。

写回时，运行时提交门同时接收产物生命周期、记忆提交对象、质量门结果、答案已采用状态、输入验证器和输出验证器。所有条件通过，候选 **ArtifactRef** 才被标记为已验证，**MemoryRef** 才进入已提交状态并保留回放类别；任一条件失败，产物转为已失效，记忆仅保留为不可直接回放的候选对象或被显式失效，并生成资源结算与失效记录。读侧兼容门与写侧提交门因此构成闭环：前者防止旧对象错误进入当前任务，后者防止当前错误变成未来历史。

3.5 基于提示词布局的前缀复用机制

在多智能体任务中，不同角色经常需要重复读取同一批长证据。结构化通信能够减少智能体之间的控制消息和重复对象传输，但当证据进入模型推理阶段时，仍然需要作为提示词的一部分提交给模型服务。如果每个角色的提示词都从不同的角色指令和动态参数开始，即使后面包含相同证据，推理引擎也难以将其识别为相同的 KV 缓存块，需要重复执行预填充计算。

为减少这部分重复计算，**StateBus** 增加了基于提示词布局的前缀复用机制。系统首先取得参与复用角色各自可见的证据集合，再根据来源定位、内容摘要和可见范围计算共同授权证据。只有同时对参与角色可见且内容一致的证据，才能进入共享前缀；角色专有证据、角色指令、任务参数、工具信息和运行标识仍保留在角色后缀中。如果共同证据为空、来源定位冲突或内容摘要不一致，系统不构造共享前缀，而是回退到独立提示词模式。

共享提示词模式将共同证据放在提示词起始位置，形成共同证据前缀与角色专属后缀的稳定结构。不同角色之间的差异被推迟到共同前缀之后，从而使最终 **Token** 序列能够从位置 0 开始形成相同的完整缓存块。独立提示词则保持角色指令和动态字段优先的原始布局，角色差异会较早出现，因此后续相同证据通常无法形成相同的前缀缓存块。

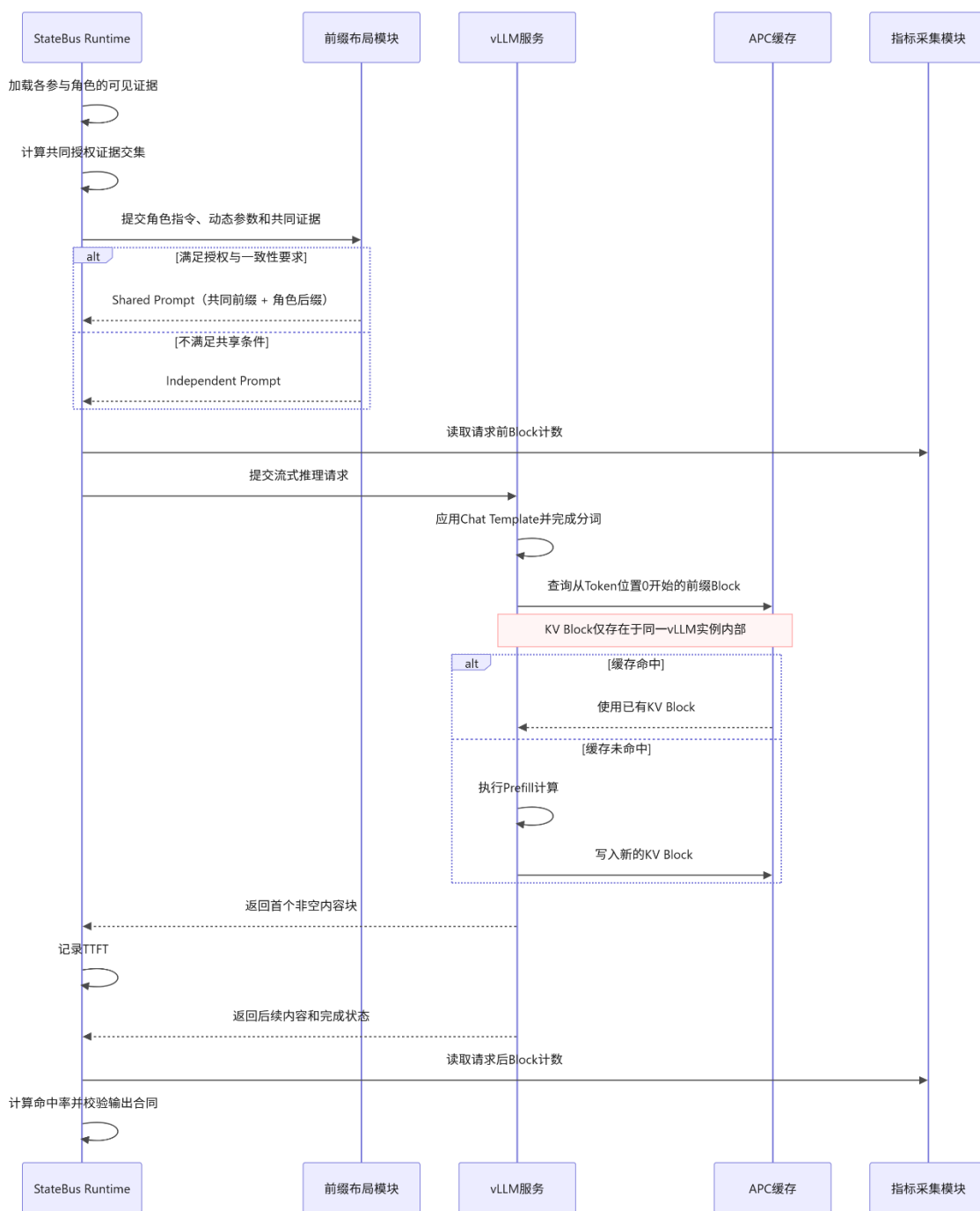


图 3-7 基于提示词布局的前缀复用时序

如图 3-7 所示，前缀布局在请求提交之前完成。StateBus 将整理后的完整提示词提交给 vLLM 服务，vLLM 随后应用对话模板并完成分词编码，再由 APC 查询从 Token 位置 0 开始的前缀缓存块^[7]。若缓存中已经存在身份一致的完整缓存块，vLLM 直接使用已有 KV 缓存块；若未命中，则正常执行预填充计算并写入新缓存块。首个非空流式响应到达后，运行时记录首 Token 时延；请求结束后，再根据前后计数器差值计算本次请求区间的缓存块命中情况。

前缀复用还受到运行环境和调度条件约束。可复用请求需要使用相同的 vLLM 实例、模型、分词器、对话模板和兼容的缓存环境。系统记录共同前缀哈希、提示词哈希、角色后缀身份、缓存块计数和运行环境信息，并对最终请求的公共 Token 前缀与缓存块对齐情况进行审计。在任务并行执行时，运行时只在依赖已经满足的就绪任务集合中优先选择具有相同语料前缀的任务，不会为了提高缓存命中率改变任务依赖图或提前执行下游步骤。

需要强调的是，该机制优化的是相同长前缀的重复预填充计算，而不是减少提交给模型的输入内容。StateBus 不读取、导出、传输或持久化 vLLM 内部的 KV tensor，KV Block 的创建、保存、命中和淘汰均由同一 vLLM 实例内部完成。因此，前缀复用与 SemanticStateRef 属于不同机制：前者减少模型推理阶段的重复计算，后者负责独立进程之间数值状态的显式传递。

3.6 受限 Python CodeAct、结构化 DSL 与可信产物

执行者支持受限 Python CodeAct^[8] 和结构化 DSL 两类执行表示。两类任务都经过规划者、检索者、执行者、总结者四角色链，区别只发生在执行者如何表达计算。对组合清洗、统计和跨表逻辑，模型可以根据批准输入生成 Python；对于字段稳定、操作可枚举的任务，计划可以选择 DSL capability，并给出结构化参数。DSL 不是预先写好的最终答案，Python 也不因为由 LLM 生成就自动获得执行权限。

受限 Python 路径由执行模块组织为生成、审计、物化、隔离执行、验证和提交六个阶段。生成请求只包含 ApprovedPlan 中该步骤的目标、输入对象清单、逻辑文件名、允许库、输出结构和策略要求，不暴露宿主机绝对路径。模型返回后，运行时提取单一 Python 程序，并分别计算生成请求哈希、原始响应哈希和源码哈希；因此后续策略、执行记录与产物能够精确指向哪一版代码，而不是只保存最终标准输出。

受限 Python 路径从 CapabilityGrant 开始。Grant 绑定任务、会话、步骤、执行尝试、允许能力、输入 Ref、输出合同、工作区、运行预算和有效时间；运行时再用能力描述核对所属角色与质量验证器。模型只能看到被物化到当前工作区的逻辑输入名，无法通过提示词扩大路径范围。生成源码先经过 AST 与策略审计，系统拒绝动态求值、动态执行、动态导入、危险模块、路径穿越和未授权文件访问，并限制语法树规模、调用、循环与输出位置。策略失败时程序不会启动，源码哈希和拒绝原因被保留。

代码修复也被纳入同一信任边界。策略修复、运行时修复和质量修复各有独立预算，修复请求只能携带结构化错误摘要和既有合同，不能增加输入 Ref、能力或输出路径。每次修复都会产生新的源码哈希，并从 AST/策略审计与就绪性检查重新开始；上一版失败源码和报告仍保留在当前执行尝试的工作区。达到预算后明确失败，不把有问题的程序切换到无隔离执行，也不把部分输出拼接成正式产物。

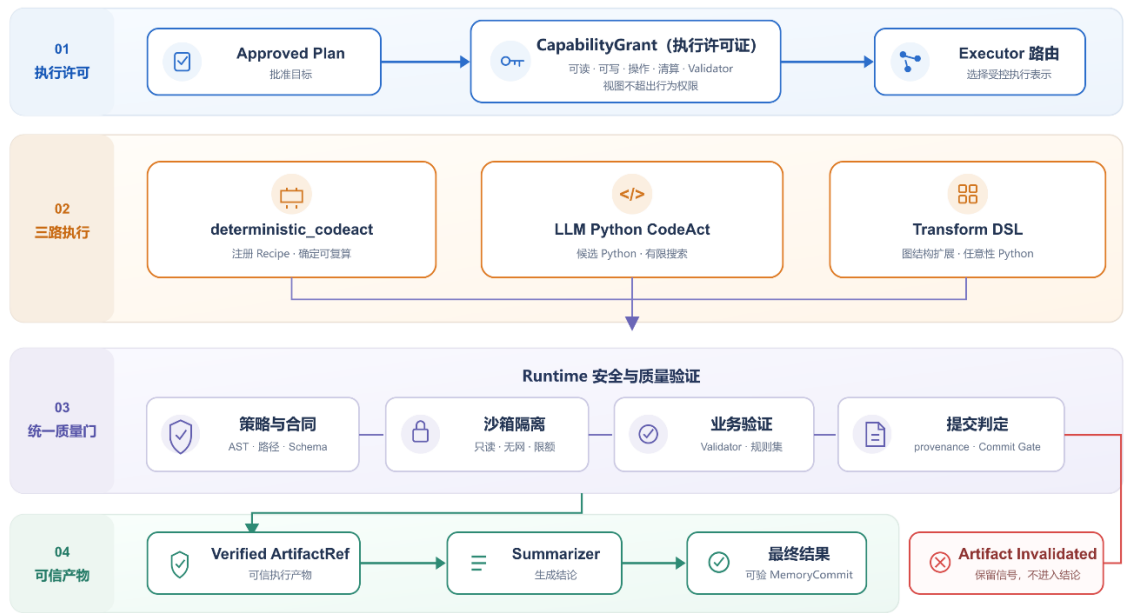


图 3-8 受限 Python CodeAct 执行过程

正式受限 Python CodeAct 在执行前必须通过 bubblewrap 隔离环境就绪性检查^[9]。该检查不仅确认命令是否存在，还实际验证进程采用非特权身份、网络不可用、批准输入只读、输出目录可写、仓库与其他任务工作区不可见。执行时，生成的受限 Python 程序以只读方式挂载到沙箱程序目录，输入目录只读，只有输出目录可写，并设置超时、CPU、地址空间、文件大小、打开文件数和进程数限制。正式路径不会在 bubblewrap 不可用时静默改为无隔离运行。

沙箱执行模块使用显式参数列表启动 bubblewrap，不通过命令行解释器拼接指令；运行环境只保留必要变量，并隔离网络、进程标识空间、挂载点和用户权限。Python 子进程同时受到超时与资源配额约束；超时或资源超限都会形成结构化退出原因。执行模块只收集限定长度的标准输出和标准错误，并把沙箱类型、策略摘要、运行身份、退出原因与执行耗时写入结果合同。操作系统隔离只回答代码能访问什么，不能回答计算是否正确，因此不能替代后续业务验证器。

工作区管理模块以任务、步骤和执行尝试建立独立目录，保存输入对象清单、程序、策略报告、标准输出和标准错误、输出对象清单、验证器报告和资源结算记录。进程返回 0 后，输出仍处于候选状态：运行时检查文件是否位于当前输出目录、是否为符号链接、哈希值与输出结构是否一致，再由能力验证器对业务字段、预期事实和必要的独立重算进行检查。全部通过后，产物生命周期管理模块才将对象标记为已验证；否则标记为已失效，保留诊断信息但禁止总结者和记忆存储模块消费。

DSL 路径使用注册 opcode、参数结构约束、字段类型和输出合同，由确定性解释器执行。它不需要承担 Python 的全部语法和导入风险，但同样只能访问批准输入，同样进入工作区，并经过输出验证器与提交门。项目正式能力任务中 18 个使用受限 Python CodeAct、7 个使用结构化 DSL，反映的是任务执行表示的差异，不代表其中 7 个任务绕过了 LLM 角色链。

当前 DSL 只开放选择、重命名、过滤、排序、截取、分组聚合、安全派生、期间比较、键连接、异常检查和结论投影等注册操作。结构化程序验证模块在执行前检查结构版本、授权输入 Ref、操作数、行列与输出字节预算、字段是否存在，以及连接操作右侧 Ref 是否获准，并拒绝路径、文件、表达式和 Python 等可能引入解释器或文件访问面的参数。解释器输出仍会形成候选 ExecutionArtifactRef，再经过与 Python 路径相同的质量报告和已验证提升，因此 DSL 的确定性来自受限指令集，而不是绕过产物可信链。

3.7 软件界面、前后端与部署

StateBus Studio 面向中文项目审阅与交互场景，只保留实验与证据和任务运行两个一级页面。数据集详情、智能体全记录、代码、StateRef、MemoryRef、产物和质量报告通过详情面板或二级视图展开，避免把产品做成堆满脚本按钮的运维页面。界面不展示模型隐式思维过程，而是展示可以审计的输入合同、结构化输出、生成程序、对象哈希、消费回执和验证器结果。

Evidence Center 读取固定证据快照文件。顶部固定展示质量、总 Token、线路传输字节和 10 个任务的总耗时，随后展开结构化通信、非文本状态、共享记忆和能力覆盖。页面只负责渲染固定字段，不在浏览器中重新计算正式结论；临时 Run 存入独立历史，不能覆盖快照。这样，本文与系统界面使用相同的统计分母、单位和数字。

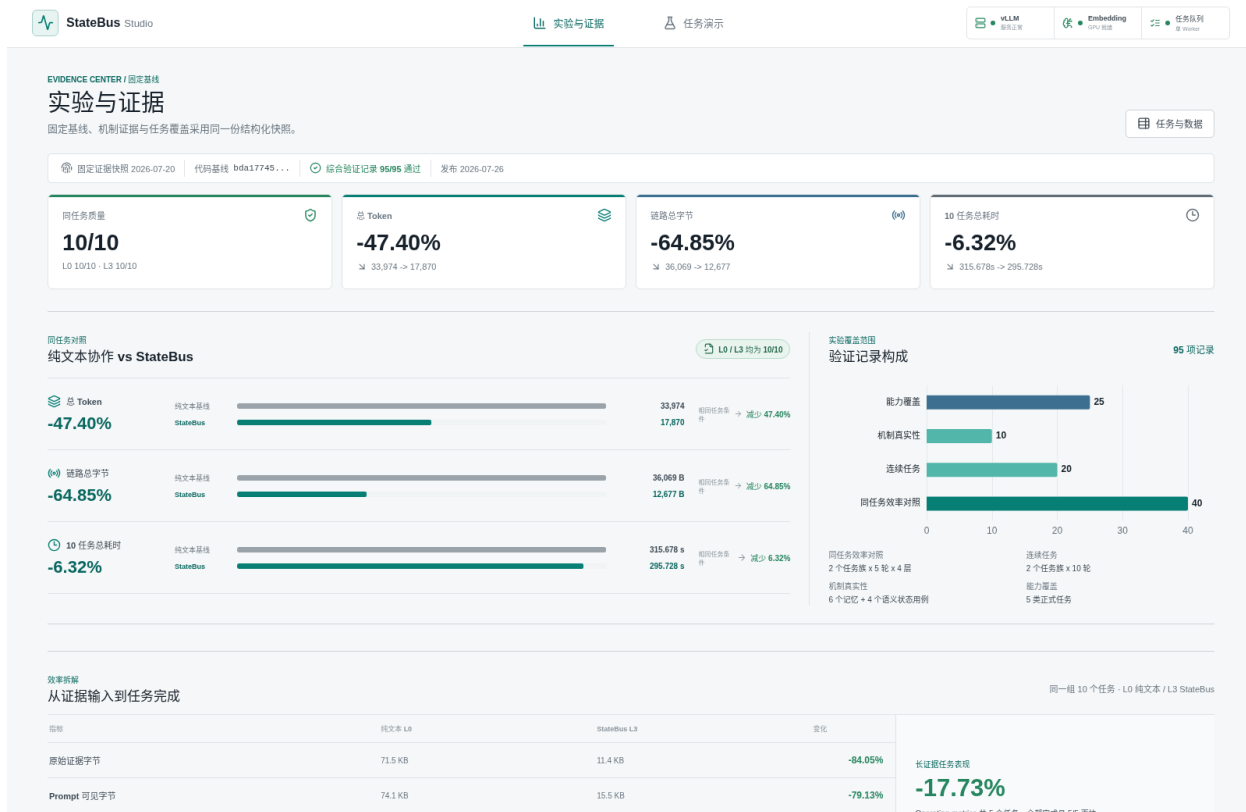


图 3-9 Evidence Center 固定证据页

Live Studio 从白名单选择快速任务、连续场景或正式实验。任务启动后，上半区以规划者、检索者、执行者、总结者和类型化对象节点构成动态流程，当前智能体由真实事件高亮，对象标签在完成交接后沿连接线移动；下半区紧凑呈现当前智能体的输入、转换和输出。选择执行者时可以查看完整生成的 Python 程序或 DSL 指令、策略、bubblewrap、产物与验证回执，选择检索者时可以查看查询、候选、EvidencePack 和 StateRef 消费。任务失败会冻结在发生错误的角色，并显示中文阶段和根因，不伪造后续节点。

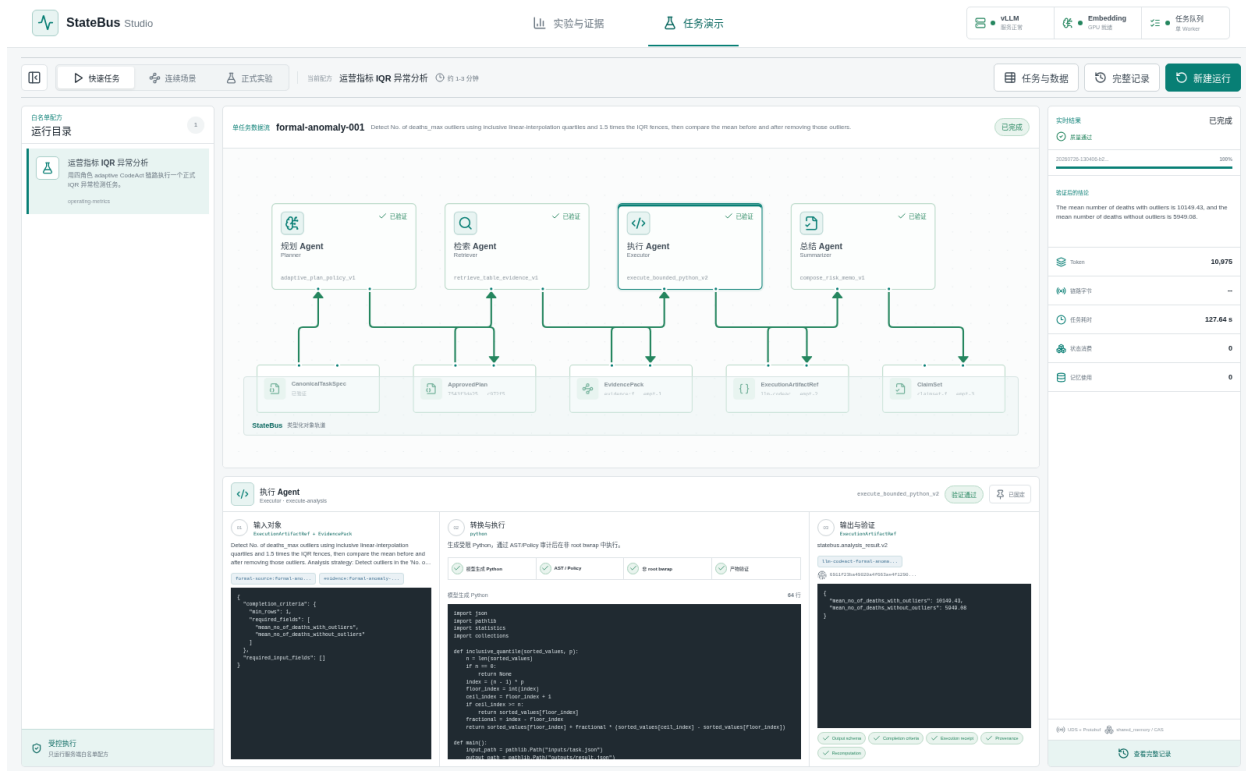


图 3-10 Live Studio 已完成任务的执行者视图

前端位于 `studio-ui/`，使用 React、TypeScript、Vite、React Flow、ECharts 和 Lucide。页面维护当前运行、流程节点、选中角色和详情状态，通过 SSE 接收结构化事件；网络中断不会取消后端任务，刷新后可以按运行 ID 恢复。任务完成后新建运行只重置当前画布和选中状态，不删除历史运行，新的 SSE 订阅也会忽略上一运行的晚到事件。

Studio 后端采用 FastAPI 后端框架。任务目录管理模块负责读取任务和数据目录，白名单配方映射模块将前端配方标识转换为固定运行参数，作业调度与恢复模块负责单工作进程队列、独立进程组、事件索引、持久化、取消和恢复，任务流重建模块根据运行摘要、调用轨迹、生成程序与执行产物恢复四角色视图。浏览器不能提交任意命令行指令、路径或 Python 参数，也不能启动或重启 vLLM；创建运行时只提交白名单配方及其明确允许的字段。

创建作业前，API 会并行检查既有 vLLM 服务健康状态、嵌入模型目录与设备、规划者执行组件可导入性、环境变量 `PYTHONPATH` 是否包含项目路径，以及单工作进程队列是否存活；任一关键项失败，开始运行会返回中文阶段原因，而不是先排队后崩溃。白名单配方映射模块直接构造启动参数列表，为每项配方固定运行器、任务套件、角色路径、嵌入模式、本地套接字和输出目录，不经过命令行解释器。作业调度模块为

每次运行建立独立目录和进程组，将长任务的标准输出和标准错误完整写入日志，持续读取结构化运行时事件并分配递增的 SSE 事件编号。取消操作只向当前运行的进程组发送信号，不触碰既有 vLLM 或其他历史目录。

任务详情的恢复也采用受限读取。任务流重建模块只在当前运行根目录内寻找运行摘要、规划轨迹、生成程序、验证器报告和执行产物，解析前同时检查路径归属、文件类型与大小上限，不能通过产物中的路径字段读取仓库其他位置。服务重启时，作业管理模块根据作业状态文件和 JSONL 事件日志恢复终态和序号，前端刷新后以运行 ID 和最近事件编号继续订阅。新建运行清空的只是页面中的当前运行、选中角色和动画状态，历史证据与后端任务不被删除，也不会让上一运行的晚到事件覆盖新流程。

详细接口清单见附录 A 的 Studio 主要接口。

交付环境采用 Docker 单容器与 openEuler 24.03 LTS-SP3。容器内包含 StateBus 运行时、四类智能体工作进程、Studio 接口与静态资源、UDS、状态池、任务工作区和记忆存储模块，模型、缓存、Run、日志及工作区挂载到用户目录。启动脚本显式设置项目的 Python 模块搜索路径；Embedding 使用配置的容器内设备，模型服务只复用已健康运行的 vLLM，不由网页或启动脚本重启。容器提供交付环境一致性，CodeAct 的最小执行边界仍由内部 bubblewrap 隔离配置单独承担。

Studio 默认通过 <http://127.0.0.1:8769/> 访问。正式长实验写入独立 Run 目录并由汇总器读取，Studio 只订阅结构化事件和索引；因此交互运行、正式实验和固定证据虽然共享代码实现，却保持各自清晰的数据边界。

第 4 章 实验测试

4.1 测试环境说明

本项目采用 Docker 单容器作为统一测试与交付环境，基础系统为 openEuler 24.03 LTS-SP3^[10]。容器内集成 StateBus 运行时、规划者、检索者、执行者、总结者四类智能体工作进程、状态池、共享记忆、工作区和验证组件，从而减少主机操作系统、Python 依赖及模型运行环境差异对测试结果的影响。

项目测试分为两类环境：一类为不依赖模型服务和 GPU 的确定性环境，用于协议、合同、状态对象、共享记忆、验证器和前端等回归测试；另一类为使用本地模型和 GPU 的真实模型环境，用于验证真实检索、非文本状态跨进程消费、共享记忆复用和 CodeAct 执行链路。

表 4-1 环境配置

配置项	测试环境配置
操作系统	openEuler 24.03 LTS-SP3 Docker 容器
Python 版本	Python 3.11（测试基线）
GPU 运行时	NVIDIA Container 运行时；通过 STATEBUS_EMBED_DEVICE 指定设备
智能体通信	Unix 域套接字（UDS）和 subprocess transport
非文本状态存储	共享内存；必要时使用 mmap 或 memfd
CodeAct 隔离	bubblewrap（bwrap）
容器网络	主机网络模式
共享内存	1 GiB
目录挂载	模型、缓存、运行记录和工作区统一挂载到/statebus

容器内主要目录包括：/statebus/models 用于保存模型文件，/statebus/caches 用于保存依赖与模型缓存，/statebus/runs 用于保存测试运行记录，/statebus/work 用于状态池和临时运行数据，/statebus/workspaces 用于受控 CodeAct 工作区。

测试使用角色模型和 Embedding 模型，分别承担语言决策与语义编码任务，具体配置如下。

表 4-2 模型配置

模型类别	模型名称	主要作用
角色模型	Qwen3-32B	承担规划者、检索者、总结者推理，并参与执行者代码生成
嵌入模型	Qwen3-Embedding-0.6B	编码查询、证据和记忆候选，支持语义检索与状态消费

角色模型服务默认通过 `http://127.0.0.1:53334/v1` 访问。测试过程只复用并检查已有 vLLM 服务状态，不负责启动、停止或重启模型服务。嵌入模型默认存放于 `/statebus/models/Qwen3-Embedding-0.6B`，并由容器内可见的 GPU 执行推理。

在部分稳定性和效率测试中，执行者使用确定性的受限 Python CodeAct 路径，以减少自由代码生成带来的随机性；在能力覆盖测试中，执行者使用受限 Python CodeAct 或结构化 DSL，并依次经过策略检查、沙箱执行和产物验证。

4.2 实验任务设置

本项目根据验证目标设置同任务配置对照、连续任务、机制真实性用例和能力覆盖四类正式实验，共完成 40 次同任务配置对照、2 个 10 轮连续任务、10 个机制真实性用例和 25 个能力覆盖任务。同任务配置对照用于比较纯文本协作与完整 StateBus 在 Token、通信载荷、执行时间和结果质量方面的差异；连续任务用于验证共享记忆在跨轮依赖场景中的候选召回、兼容校验和实际使用；机制真实性用例用于检查结构化控制消息和非文本状态是否发生真实传递、跨进程消费并改变下游行为；能力覆盖任务用于验证系统处理不同数据形态和分析操作的能力。

连续任务由运营与财务两种各 10 轮的顺序任务链构成。运营任务链以疾病统计 CSV 文件和逐小时天气 CSV 文件为输入，覆盖表结构与缺失率分析、条件聚合、极值定位、四分位距（Interquartile Range, IQR）异常检测、跨数据集策略迁移、字段漂移解析、数据清洗和长程来源汇总；财务任务链以紧凑合成的跨期财报 Markdown 文本为输入，覆盖指标抽取、季度差值、跨公司对比、字段别名解析和三季度趋势汇总。后续轮次通过前序依赖和复用合同引用已经验证的事实、策略或执行产物，因此 20 轮任务构成两条连续工作流，而不是 20 个相互独立的问题。

能力覆盖实验包含五类共 25 个独立任务，其中财报指标抽取 8 个、多期趋势分析 5 个、跨表关联分析 5 个、条件聚合 4 个、异常检测与清洗 3 个。输入由离线财务语料、

跨期财报 Markdown、疾病统计 CSV 和天气 CSV 构成。每个任务均具有固定的 CanonicalTaskSpec、输入参数、产物结构和 expected facts，任务之间不编码跨轮依赖。受限 Python CodeAct 或结构化 DSL 由运行时根据任务操作、能力授权和运行条件选择，并非任务注册表预先指定的固有类别。

正式任务的通过判定同时检查任务合同、产物结构、确定性事实以及来源与生命周期信息。只有结构化产物满足规定字段，关键结果符合精确匹配或数值容差要求，并且产物引用、来源定位和消费记录完整时，任务才能通过验证器。共享记忆实验进一步区分候选召回、兼容校验和实际消费，避免将检索到相似历史内容直接视为有效复用。各类任务的具体输入、跨轮依赖、预期产物和确定性验证规则见附录 C 正式实验任务与数据集说明。

4.3 结果总览

表 4-3 主要实验结果总览

验证维度	主结果	对应样本	结论定位
任务质量	95/95 正式任务执行通过	效率、连续、真实性和能力覆盖	全部机制结论的质量基线
总 Token	33,974 → 17,870（降低 47.40%）	10 个同任务对照	完整链核心结果
线路传输字节	36,069 → 12,677（降低 64.85%）	10 个同任务对照	完整链核心结果
总耗时	315.678 s → 295.728 s（降低 6.32%）	10 个同任务串行对照	完整链核心结果
结构化控制	50 条消息；control bytes 降低 83.05%	纯文本与结构化协议对照	通信专项
非文本状态	9/9 跨进程消费并改变选择	3 个语义状态任务	状态专项
共享记忆	实际使用为 7/20（35%）	两组各 10 轮	记忆专项

验证维度	主结果	对应样本	结论定位
能力覆盖	25/25；18 个受限 Python CodeAct，7 个 结构化 DSL；0 回退	5 类分析任务	系统完整性

上述结果总览汇总了同任务配置对照、连续任务、机制真实性用例和能力覆盖任务的主要结果。前缀复用、LogitState 受控重试和显式 KV 状态继承的实验结果分别在后续小节报告。

4.4 总体实验结果

本实验用于验证 StateBus 作为完整运行时投入使用后，能否在不减少协作步骤、不降低结果质量的条件下，降低四角色主链的通信与模型上下文开销。实验选取 10 个相同任务，分别采用纯文本协作和完整 StateBus 两种配置串行执行。两种配置使用相同的模型、输入数据、任务合同、角色分工和质量验证规则，并保持智能体消息数量一致；区别在于，完整 StateBus 启用了结构化控制、证据引用化、角色最小可见输入和产物引用传递。实验分别统计控制面字节、线路传输字节、模型可见原始证据、提示词 Token、总 Token 和任务耗时，用于评价多项机制共同作用下的端到端效率，而不是单独衡量某一项机制的效果。只有两种配置均通过质量验证，相关开销差异才计入最终结果。



图 4-1 同任务完整链效率对比

表 4-4 十个同任务完整链整体对比

指标	纯文本协作	完整 StateBus	变化
任务数 / 质量通过	10 / 10	10 / 10	质量保持
智能体消息数	50	50	0
控制面字节	25,196	5,357	-78.74%
线路传输字节	36,069	12,677	-64.85%
LLM 可见原始证据字节	73,266	11,687	-84.05%
提示词可见字节	75,926	15,847	-79.13%
提示词 Token	29,876	13,885	-53.52%
总 Token	33,974	17,870	-47.40%
10 任务总耗时	315.678 s	295.728 s	-6.32%

如图 4-1 所示，在消息数和质量均保持不变的条件下，完整 StateBus 将总 Token、线路传输字节和 10 任务总耗时分别降低 47.40%、64.85%和 6.32%。收益主要来自原始证据引用化、提示词与上下文收敛和控制帧压缩。

4.5 结构化通信专项

本实验用于单独验证结构化控制面相较于纯文本消息在通信开销和协议可靠性方面的作用，避免将完整链中的证据引用、状态传递和提示词压缩等机制混入比较。实验围绕相同协作过程构造两组各 50 条语义等价的控制消息：纯文本组使用自然语言描述任务动作、参数和执行状态，结构化通信组使用类型化 Protobuf 消息表达步骤标识、目标角色、操作参数、对象引用和执行回执。两组保持消息数量和业务含义一致，分别统计控制面序列化后的总字节数，并检查消息能否被正确解析和处理。该实验主要回答在不减少协作事件的情况下，将重复文本改为固定字段和引用句柄，能否有效降低线路载荷，同时保持消息语义明确、可校验和可追踪。



图 4-2 结构化通信对比

表 4-5 控制协议专项结果

指标	纯文本	结构化协议	变化
任务质量	10/10	10/10	保持
智能体逻辑消息	50	50	0
控制面字节	25,196	4,270	-83.05%
线路传输字节	36,069	11,200	-68.95%

相同 50 条逻辑消息且正确率均为 10/10 时，结构化控制将控制面字节降低 83.05%，线路传输字节降低 68.95%，说明收益来自载荷收敛而非减少协作轮次。

4.6 非文本状态专项

4.6.1 StateRef 机制效果

为验证 StateRef 是否实现了非文本状态的真实传递与消费，本实验选取 4 个保留任务，其中包括两个长文本任务、一个路由对照任务和一个文本与表格联合分析任务。3 个任务启用 SemanticStateRef，另 1 个任务不生成语义状态，作为路由对照。

在启用 SemanticStateRef 的任务中，检索者分别生成查询嵌入向量、候选嵌入矩阵和稠密选择矩阵，并以 32 位浮点数的形式写入状态池。控制面仅向下游传递 StateRef，

不内联数值内容。随后由独立进程中的消费者读取对应状态，根据数值结果完成候选排序和证据选择。实验记录生产者与消费者进程标识、数据类型、矩阵形状、编码器签名、候选编号、选择得分、消费回执以及状态释放字节数。

StateRef 被认定为有效使用，需要同时满足以下条件：生产者与消费者属于不同进程；消费者实际读取数值对象；数值状态参与并改变下游证据选择；入选证据进入后续任务链；任务通过外部质量验证；状态在任务结束后完成释放。

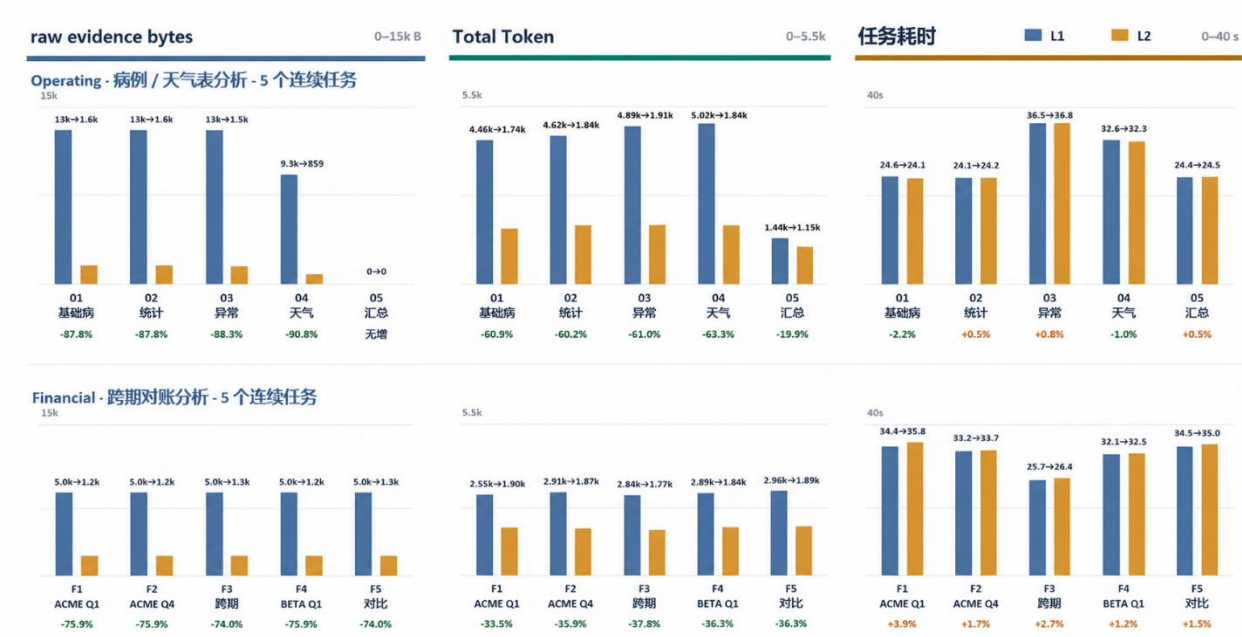


图 4-3 非文本状态实测链

如图 4-3 所示，完整 StateBus 在两组任务中均明显减少了大模型可见的原始证据字节数和总 Token。Operating 任务中，前 4 个证据密集型任务的原始证据字节数降低约 87.8%~90.8%，总 Token 降低约 60.2%~63.3%；不需要读取原始证据的路由任务变化较小。Financial 任务的原始证据字节数降低约 74.0%~75.9%，总 Token 降低约 33.5%~37.8%。两种方式的单任务耗时整体接近，部分任务略有上升或下降，说明引用化传递主要减少了重复上下文和 Token 消耗，但端到端耗时仍受到模型生成、进程调度和结果验证等环节影响。

4.6.2 LogitState 受控重试验证

普通任务通常会让模型直接给出较明确的候选选择，难以观察候选概率状态是否真正改变了运行时行为。因此，本实验没有统计自然任务中的重试发生率，而是构造明确

任务、受控歧义任务和不可判定负例三类受控任务，分别验证判定门是否会错误触发、是否能够纠正不确定路由，以及能否在缺少合法选择时阻止业务工作进程启动。

实验共包含 12 个任务，其中 5 个明确任务从首次选择开始就提供完整合同；5 个受控歧义任务在首次选择时只提供最小角色可见信息，检测到较低概率差值后才展开决定性合同；2 个不可判定负例在重查后仍不存在唯一且合法的候选。每个任务分别以判定门关闭和 `retry_once` 模式运行，共形成 24 次配对执行。

两种模式使用相同模型、候选集合和首次角色可见信息。为降低候选别名顺序偏差，受控实验对同一次选择交换 A、B 候选绑定，并按照候选标识对齐概率。该 AB/BA 校准只用于本次机制挑战，不属于正式运行时的默认路径。外部标准答案与模型可见任务清单分离，任务期望不会进入提示词。

表 4-6 LogitState 受控重试实验结果

任务类型	数量	判定门关闭通过	Retry once 通过	判定门行为
明确任务	5	5/5	5/5	0/5 触发重查，无误触发
受控歧义任务	5	3/5	5/5	5/5 触发重查，纠正 2 个错误路由
不可判定负例	2	0/2	2/2	2/2 触发重查，错误放行由 2 次降为 0 次
合计	12	8/12	12/12	7/12 触发重查

不可判定负例中的 2/2 通过表示系统正确执行了 `fail closed`，而不是模型最终选出了可执行候选。两个负例在第二次判断后仍保持低 `Margin`，运行时因此没有生成工作进程放行判定，也没有启动后续 `CodeAct`、结构化 DSL 或业务工作进程。

在 `retry_once` 模式下，12 个任务共形成 19 次判定门状态判断，包括 5 次明确任务首次判断、10 次受控歧义任务两阶段判断以及 4 次不可判定负例两阶段判断。19/19 个 `LogitState` 均由不同于生产者的独立 `PID` 消费，19/19 完成物理状态释放并留下生命周期记录。两候选实验中的单次载荷由两个候选概率和一个 `other_mass` 组成，共 12 B；全部状态载荷合计 228 B。该字节数只表示 `float32` 概率载荷，不包含 `Protobuf` 控制消息、`sidecar` 和进程调度开销。

受控重试同时带来了额外模型调用。判定门关闭模式共调用 vLLM 24 次、消耗 6,110 Token；retry_once 模式共调用 38 次、消耗 9,952 Token，调用次数和 Token 分别增加 58.33%和 62.88%。增加的开销主要来自实验中的 AB/BA 双请求，以及 7 个低 Margin 任务产生的第二阶段选择。因此，本实验用于验证数值状态驱动的执行授权和失败阻断，不用于证明 Token 或时延收益。

实验结果表明，LogitState 不只是被发布和读取的数值对象，而是实际参与了运行时决策：明确任务没有发生误重试，歧义任务在数值判定门触发后达到 5/5 验证通过，不可判定负例的错误工作进程放行由 2 次降为 0 次。

4.6.3 显式 KV 状态继承实验

为验证显式 KV 状态继承在完整多智能体主链中的实际效果，实验将该机制接入规划者、检索者、执行者、CodeAct 和总结者组成的完整任务流程。实验使用单个 Qwen3-32B vLLM 工作进程，在两份离线运营报告上构造 10 个不同的指标抽取任务。每个任务均包含相同的规划、检索、执行、产物验证和总结步骤，KV 机制只作用于执行者到总结者之间的共享前缀计算，不改变任务合同、CodeAct 执行路径和最终质量门。

实验设置完整重算和 KV 状态继承两种条件。完整重算条件下，执行者不保存 KV 状态，总结者需要重新提交并计算完整的 4096 Token 共享前缀及自身新增的后缀输入；KV 状态继承条件下，执行者在预填充阶段捕获共享前缀对应的 KV 状态，总结者只提交不透明 KV 状态句柄和新增后缀输入，由同一工作进程恢复已有状态并仅计算新增部分。两种条件使用相同任务、相同证据、相同提示词逻辑、相同温度参数和随机种子，APC、语义裁剪及其他复用机制均关闭。

实验先串行执行 10 个完整重算任务，再串行执行 10 个 KV 状态继承任务，每个阶段开始前进行一次完整主链预热，预热结果不计入统计。性能指标包括总结者实际计算的预填充 Token 数、首 Token 时延、请求体字节数、执行者与总结者合计耗时以及完整主链耗时。同时记录 KV 状态捕获、加载、释放、继承 Token 数、回退次数和最终注册表状态，用于确认性能变化确实来自 KV 状态继承，而不是缓存命中或静默回退。正确性通过任务质量门、必需字段和结构化产物核心内容进行判断，不要求两次生成结果的 Token 完全一致。

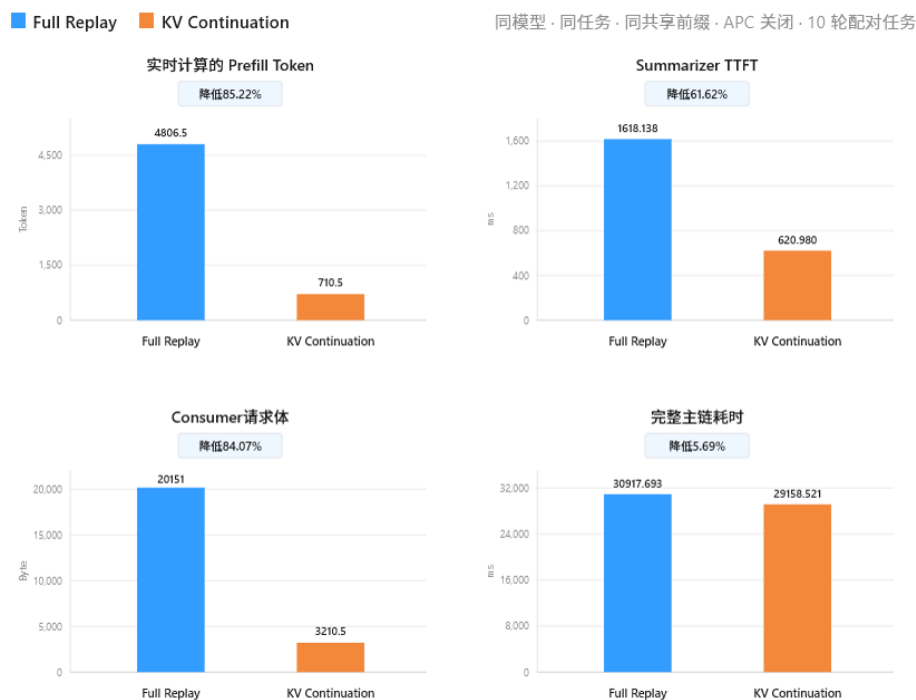


图 4-4 显式 KV 状态继承实验结果

如图 4-4 所示，显式 KV 状态继承机制首先降低了总结者侧的重复计算。KV 条件下，总结者实际计算的预填充 Token 数中位数（p50）由 4806.5 降至 710.5，降低 85.22%；首 Token 时延 p50 由 1618.138 ms 降至 620.980 ms，降低 61.62%；请求体 p50 由 20151 B 降至 3210.5 B，降低 84.07%。10 个任务在上述三项指标上均取得正向结果，说明总结者确实继承了执行者已经计算的 4096 Token 共享前缀。

4.7 共享记忆专项

本实验用于验证共享记忆能否在连续任务中实现安全、有效的跨任务复用，而不是只统计语义检索是否返回候选。实验设置两组相互关联的连续任务，每组包含 10 轮查询，使后续任务具备复用前序任务证据、分析策略或中间结论的可能。运行时依次记录候选召回、兼容校验、角色投影、实际消费和行为效果：只有候选记忆通过任务意图、数据来源、输入输出结构和运行条件检查，被目标角色实际读取并影响后续执行时，才计为一次有效复用。实验同时统计查询级复用率和不兼容候选拒绝情况，并通过负例、任务重算和重复步骤跳过等真实性用例，检查记忆机制能否在提高复用效率的同时避免错误历史内容污染当前任务。

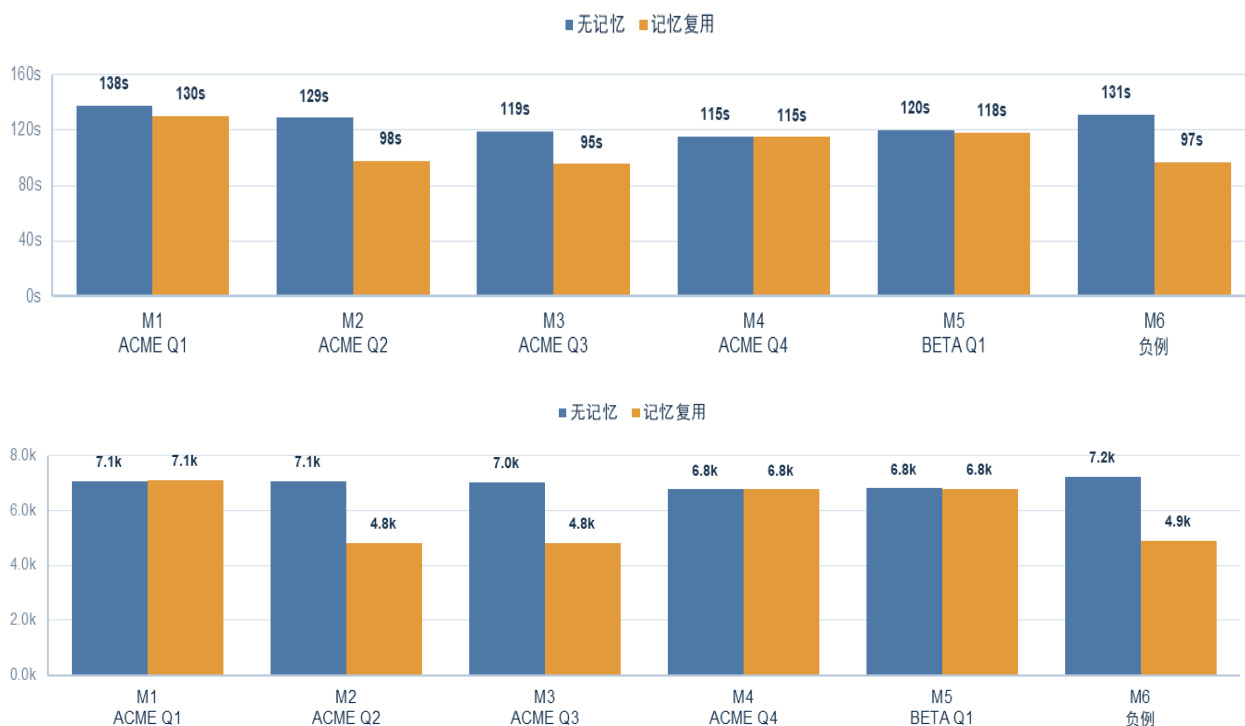


图 4-5 共享记忆自然连续任务漏斗

两组连续任务共 20 轮查询，其中 18 轮召回了候选记忆，7 轮候选通过兼容校验并被后续角色实际使用，查询级有效复用率为 35%。系统共处理 48 个候选记忆，其中 39 个因任务条件或数据约束不兼容而被拒绝，说明语义相似并不会被直接等同于可复用。两组连续任务均通过质量验证，6 个机制真实性用例也全部通过，进一步验证了不兼容记忆拒绝、当前任务重新计算以及重复步骤跳过等行为。

4.8 前缀复用专项

为验证基于提示词布局的前缀复用机制能否减少多个角色重复处理长证据产生的预填充开销，项目设置共享提示词与独立提示词两种模式进行对照。两种模式使用相同的模型服务、长证据、角色集合、生成参数和输出合同，仅改变共同证据在提示词中的位置。共享提示词将共同证据放在 Token 序列起始位置，角色指令和动态参数位于后缀；独立提示词则保留角色信息优先的原始布局。

实验使用同一 Qwen3-32B 模型和同一 vLLM 实例，设置规划者、检索者、执行者、总结者和验证者五类角色。实验包含 4 组配对，每组分别执行 5 个共享提示词请求和 5 个独立提示词请求，共 40 个请求。为降低执行顺序和缓存残留的影响，4 组实验按照共享优先、独立优先、共享优先、独立优先的顺序交替执行。同一配对内，两种模式使用

相同证据、运行标识、生成参数和 JSON 输出合同；所有请求均采用串行执行，温度参数为 0，最大生成 Token 数为 64。



图 4-6 前缀复用实验结果

如图 4-6 所示，启用前缀复用后，任务内 Block 命中率为 78.02%，每种模式 20 次请求均成功完成。整体平均 TTFT 由 2,357 ms 下降至 738 ms，降低 68.7%；平均端到端时间由 4,117 ms 下降至 2,345 ms，降低 43.0%。其中，检索、执行、总结和验证角色的 TTFT 均由约 2.3 s 下降至约 0.27 s，说明共同前缀对应的 KV Block 被后续角色请求实际命中，显著减少了重复 prefill 计算。

4.9 能力覆盖与工程稳定性



图 4-7 五类任务与执行者执行路径

表 4-7 能力覆盖任务

任务族	数量	占比	代表能力/操作
财务指标抽取	8	32%	lookup_metric
多期趋势	5	20%	compute_trend、 compute_delta
跨表关联	5	20%	compare_metric
条件聚合	4	16%	aggregate_and_extreme、 groupby_aggregate
异常检测	3	12%	detect_outliers、清洗
合计	25	100%	共 10 种操作标签

25 个能力任务全部通过，其中 18 个使用受限 Python CodeAct，7 个使用 DSL，未触发回退；两条路径均经过四角色主链和质量门。

第 5 章 实现难点说明

5.1 控制指令与非文本状态的可靠传递

多智能体协作首先要解决控制指令与中间状态如何被准确传递。自然语言可以描述任务意图，却难以稳定表达角色、步骤、能力边界、输入输出合同和对象生命周期；普通 JSON 虽然具有结构，但仍缺少强类型约束和统一身份。另一方面，embedding、候选得分矩阵等非文本状态如果被重新组织成文本，不仅会增加 Token 和编解码开销，也会丢失数值布局、编码器版本与来源信息。

StateBus 使用 UDS 与 Protobuf 构建正式控制面，并将校验分为计划策略校验、CapabilityGrant 和发送前复核三层。计划策略校验约束依赖图中的步骤、角色、能力和预算；CapabilityGrant 将任务、步骤、执行尝试、输入 Ref、输出合同与有效期绑定到哈希值；发送前再核验对象状态、对象清单和租约。这样既能确认消息格式是否合法，也能判断当前操作是否真正获得授权。

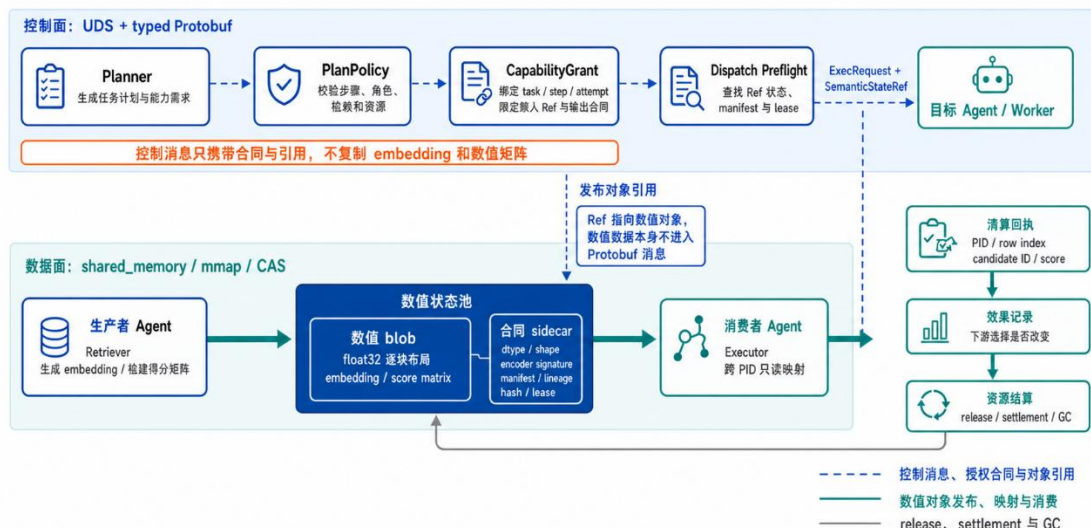


图 5-1 控制指令与非文本状态的可靠传递机制

对于非文本状态，本文使用 StateRef 泛指状态引用，其中用于传递非文本语义状态的具体对象为 SemanticStateRef。系统采用数值数据块、合同伴随元数据与 SemanticStateRef 的组合。稠密矩阵以 float32 连续布局进入共享内存，生命周期更长或需要回放的对象进入文件或 CAS；伴随元数据保存数据类型、数据形状、编码器签名、对象清单、来源链、哈希值和租约，Ref 只在控制面传递对象身份与访问条件。控制消息因此保持轻量，而数值对象仍具有可验证的结构和来源。

跨进程消费不能只证明共享内存曾被创建，还要证明另一个进程中的角色实际读取并使用了它。消费者在只读映射后返回进程 ID、行号、候选标识、得分和消费回执，运行时再关联下游选择变化、效果记录与释放事件。由此形成发布、消费、效果、资源结算和资源回收的完整闭环，既避免把对象存在当成状态生效，也防止异常退出造成租约和共享内存泄漏。

5.2 共享记忆的兼容校验与有效复用

共享记忆的核心难点不是召回相似内容，而是判断历史对象是否仍适用于当前任务。自然语言相近并不代表数据条件一致：季度、币种、聚合粒度、字段版本、数据来源、来源链或验证器任一变化，都可能使历史结论失效。若仅依据向量相似度直接复用，错误记忆会在缺少显式冲突的情况下污染当前执行。

StateBus 将稠密向量检索、BM25 关键词检索算法与标签过滤组成的混合检索仅作为候选入口，随后执行兼容门。检查覆盖任务意图、输入输出结构、数据对象清单、来

源链、运行兼容签名、质量状态和角色可见性等硬条件；关键项不一致时直接拒绝并记录结构化原因，当前任务回到正常计算路径。系统不会用更高的相似度分数掩盖语义冲突。

通过兼容门的历史对象仍不能直接广播给所有角色。系统依据目标角色重新绑定 **MemoryRef**，只暴露该角色需要的字段和来源信息，并在实际进入上下文或替代计算步骤时写入消费记录。若记忆进一步改变了执行选择、减少了重复步骤或参与最终结论，还要记录行为效果；仅被检索或展示的候选不计为有效复用。

系统据此区分候选、兼容、已消费和产生效果四个阶段。虽然表面的记忆命中率可能降低，但可以分别统计召回、兼容、消费与实际效果。未通过检查的对象仍保留原始证据和拒绝原因，供后续校正规则，而不是通过修改历史内容提高复用数字。

5.3 模型生成代码的安全执行

执行者允许模型生成 Python 程序后，系统同时面对运行安全和结果正确性两类风险。AST 白名单只能限制部分语法，无法替代操作系统级隔离；**bubblewrap** 可以限制文件、网络 and 进程能力，却不能判断统计口径、单位、数据来源与计算结论是否正确。进程正常退出只说明程序完成运行，并不意味着结果已经成为可信产物。

StateBus 将候选程序依次送入 **CapabilityGrant**、AST 与路径策略、以非 root 用户运行的 **bubblewrap** 隔离环境、只读输入挂载、最小可写工作区和资源限制。**Grant** 决定程序可以读取哪些 **Ref**、调用哪些能力以及生成何种输出；静态审计拦截危险模块、越权路径和不允许的调用；动态沙箱限制网络、进程、文件系统和运行时间。任何策略越权都直接终止，不能以修复业务逻辑为由扩大权限。

安全执行之后还要经过业务质量门。验证器检查输出结构、字段完整性、数值范围、预期事实、引用关系和来源信息，并在必要时独立复算关键结果。可修复的业务错误进入有限次数的受控修复，新源码重新计算哈希值并重走全部安全与质量检查；不可修复错误则保留源码、标准输出和标准错误、策略报告和验证报告后结束当前执行尝试。

只有同时通过安全审计和业务验证的结果，才由提交门从候选状态提升为已验证 **ArtifactRef**，并获得下游可见性。结构化 DSL 与受限 Python **CodeAct** 虽然执行路径不同，但共享相同的输入合同、输出合同、验证器和提交规则。因此，下游角色消费的是带身份、哈希值、来源和质量状态的产物引用，而不是一段未经核验的模型输出。

5.4 多智能体运行的恢复与审计

多智能体依赖图同时存在并行执行、依赖汇合、超时、重试和晚到事件，难点在于保持业务步骤身份稳定，同时避免失败执行污染新的尝试。`step_id` 表示固定的业务节点，用 `attempt_id` 表示一次具体执行；只有全部必需输入达到已验证状态时才允许分发。某一步失败后，未变化的已验证上游对象可以继续复用，旧候选产物则被冻结，并为新的执行尝试签发独立 Grant 和工作区。

所有控制帧、心跳、工作区、ArtifactRef 与运行事件都携带 `attempt_id`。即使旧工作进程在 `heartbeat timeout` 后又发送 `SuccessResult`，运行时也能识别它属于已经关闭的尝试，禁止覆盖当前状态。恢复控制器依据失败发生在计划、发送、运行还是产物验证阶段，选择拒绝、重签 Grant、局部重试或重新规划，使恢复范围从全链重跑收敛到受影响的步骤。

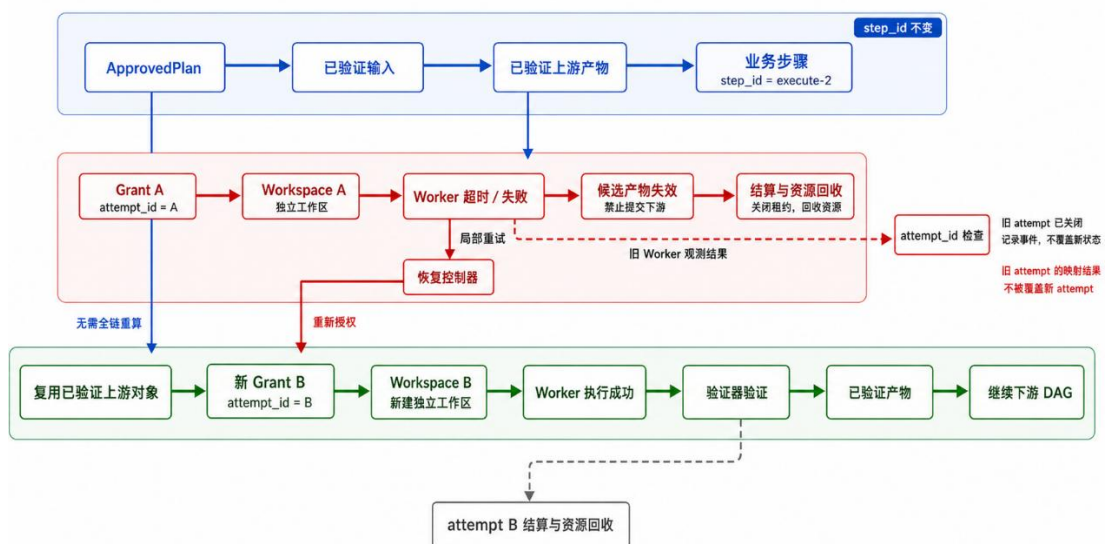


图 5-2 执行尝试失败隔离与局部恢复机制

失败处理还必须覆盖资源结算。无论执行尝试最终成功、失败、取消还是进入异常捕获状态，系统都会关闭下游可见性，处理 `StateRef` 租约、工作进程组、工作区、候选 `ArtifactRef` 和记忆写回申请，最后进入资源结算与回收。主要失败位置、处理方式和保留证据见表 5-1；这些记录既支持自动恢复，也为问题复现和责任定位提供依据。

表 5-1 主要失败路径、系统行为与保留证据

失败位置	典型原因	系统行为	主要审计证据
计划/授权	非法角色、环、预算超限或哈希/时效不符	拒绝分发；必要时受控修复	策略结果、计划/授权哈希、拒绝原因
UDS/工作进程	解帧失败、ACK/心跳超时或进程退出	标记终态并终止目标进程组	attempt、最后心跳、结构化错误
SemanticStateRef	结构约束、shape、encoder、hash 或 lease 不符	拒绝映射并释放对象	伴随元数据、Ref、校验原因、释放事件
共享记忆	intent、结构约束、lineage 或运行环境不兼容	拒绝注入并重新计算	候选排名、兼容结论、重算效果
CodeAct	AST/路径越权、bwrap 未就绪或资源超限	拒绝启动或终止沙箱	源码哈希、策略、就绪状态和输出日志
产物/结论	输出 schema、事实复算、来源或引用失败	产物失效，不交给下游或长期记忆	验证报告、候选哈希、结算记录

可恢复执行还要求不同视图使用同一份运行事实。工作进程事件、任务运行摘要、实验汇总、说明书与系统界面如果分别计算，事件增量、任务快照、失败样本和统计分母很容易发生漂移。StateBus 因此以 Run 目录为事实源，用调用轨迹、任务、步骤、执行尝试、对象标识和哈希值关联事件、回执、验证器报告与运行台账。

汇总器区分可累加事件与任务终态快照，再生成固定证据快照。本文与 Evidence Center 从该快照读取正式结果，Live Studio 只读取独立的实时运行，不直接改写固定证据。这样既保证不同呈现视图中的数字一致，也可以从汇总指标反向追溯到单个任务、单次执行尝试、具体对象及其验证报告。



图 5-3 统一证据底座与多视图审计结构

5.5 Prefix Cache 的精确命中与可归因复用

Prefix Cache（在本项目中对应同一 vLLM 实例内的 Automatic Prefix Caching，简称 APC）用于跨独立角色请求横向复用共同前缀的 KV Block。其首要难点是，语义相同并不等于前缀可命中：缓存比较发生在 Token Block 层，而不是自然语言相似度层。共同证据只有从 Token 序列 position 0 开始，并且空白、字段顺序、模板、系统提示和 tokenizer 版本均保持确定性，后续角色请求才可能复用同一批 Block；任一位置偏移或模板漂移都会使引擎执行完整 Prefill。

第二个难点是缓存归因。APC 的所有权、淘汰、容量与命中判断由推理引擎管理，应用侧不能仅凭一次 TTFT 下降就断言缓存已经生效；warmup、请求顺序和前序残留都可能干扰结果。与此同时，APC 命中后完整 Prompt 仍会发送到模型服务，请求体字节基本不变，真正减少的是引擎对已命中前缀的重复 Prefill 计算。因此，不能把 TTFT 改善表述为“少发送了同等比例的 Token”。

StateBus 通过确定性前缀编译器固定共享证据、分隔符和字段顺序，将角色指令、动态参数与输出约束放在共同前缀之后；同时记录模型、tokenizer、模板和运行环境签名。Runtime 为前缀合同和引擎消费事件建立独立记录，用于区分缓存对象是否具备复用条件、是否被引擎实际命中以及是否已完成释放。签名不一致、缓存未命中或引擎侧证据不足时，系统直接回到完整 Prefill，不改变输出合同和质量门。

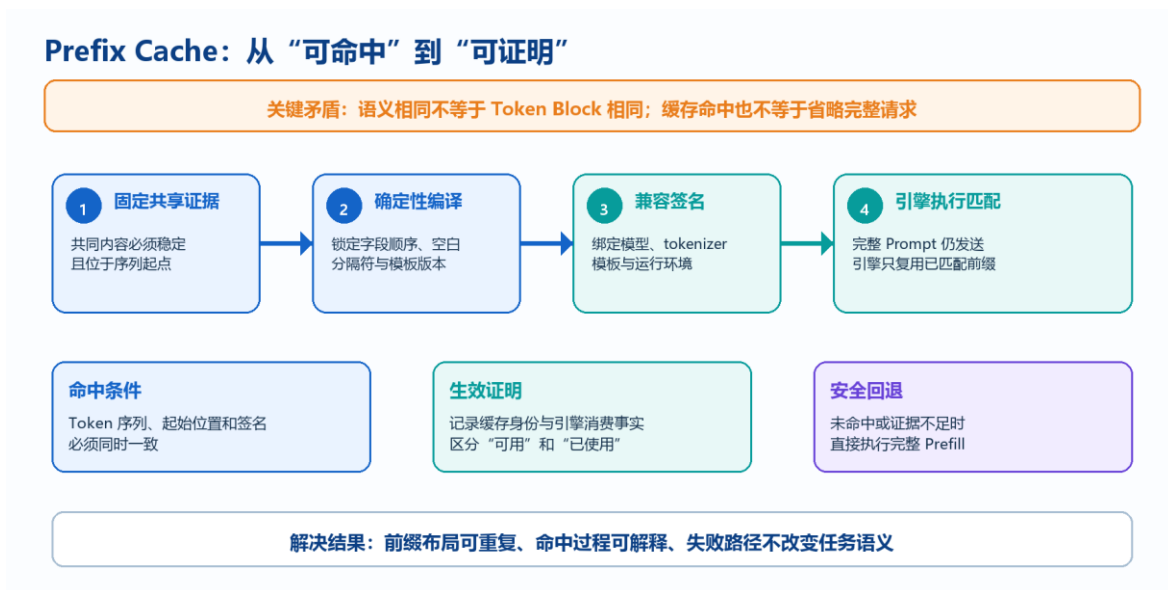


图 5-4 Prefix Cache 的确定性命中与可归因复用机制

如图 5-4 所示，StateBus 将 Prefix Cache 拆成确定性前缀生成、兼容签名、引擎命中证明和安全回退四个环节。系统首先固定共享证据在 Token 序列中的位置，再锁定模板与运行兼容条件；缓存身份与引擎消费事实分别记录，避免把“可能命中”误写成“已经生效”。任何条件不满足时，系统仍发送完整 Prompt 并执行完整 Prefill，因此缓存优化不会改变角色输入、输出合同和任务语义。

5.6 显式 KV 状态继承的兼容绑定与生命周期治理

显式 KV 状态继承面向同一任务内 Executor 到 Summarizer 的纵向续接。与 Prefix Cache 由引擎自动匹配不同，显式 KV 交接的是推理引擎内部的多层 Paged KV 神经状态。该状态体积大、布局复杂，既不能序列化进 Protobuf，也不能依靠“文本相近”近似加载；Token 序列、模型、engine、generation、KV layout、tokenizer 摘要、dtype 和 block size 任一不一致，都可能导致错误续接、缓存布局冲突或静默结果偏移。

StateBus 将 KV 保留为 engine-local handle，只在控制面传递句柄身份和访问合同。句柄绑定 task、request、step、attempt、父 Token 摘要、模型与引擎版本、generation、布局签名、有效期和容量占用；Runtime 只有在兼容签名逐项一致且对象状态为 READY 时才允许下游消费。这样既避免搬运大尺寸 KV 状态，也防止旧任务、旧 attempt 或其他引擎实例误用同一地址空间中的缓存对象。

KV 句柄按 PREPARING、READY、CONSUMING、CONSUMED、RELEASED 状态机推进，并采用 one-shot 消费。调度器返回 scheduler proof 只能证明加载请求被接

受，Worker 还必须返回 forward proof，证明真实模型前向过程继承了父序列；缺少任一证明均按未命中处理并执行完整 Prefill。无论成功、失败、取消还是超时，finally 路径都会执行 release、资源结算与 GC，并由 TTL 和容量门限制悬挂句柄。系统路由时优先使用已经 READY 的显式 KV，否则才允许 Prefix/APC 或完整 Prefill，避免对同一前缀区间重复计算收益。

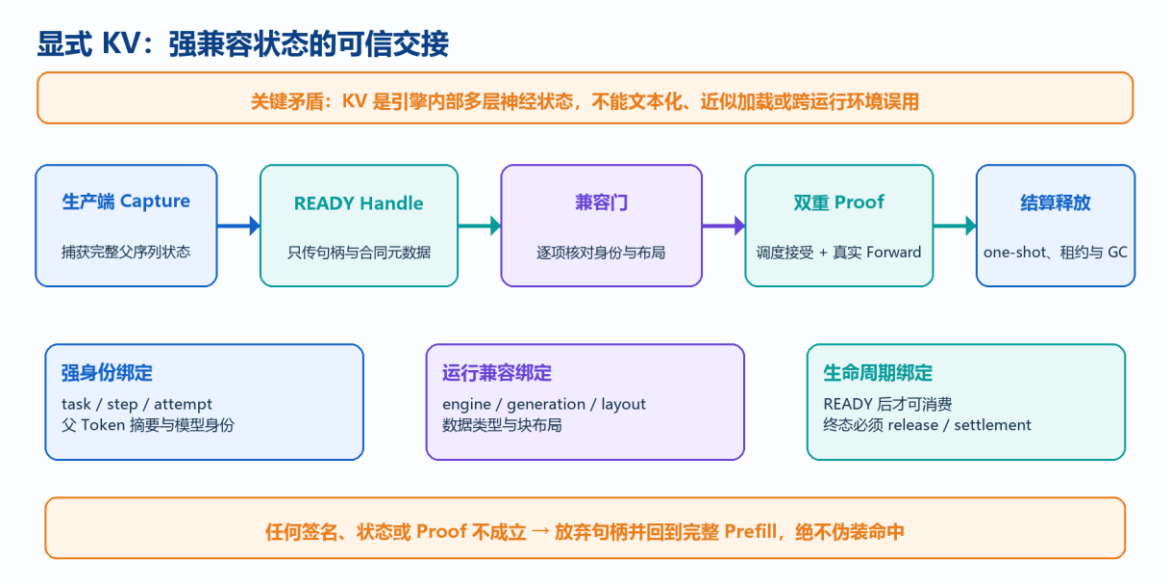


图 5-5 显式 KV 状态继承的捕获、证明与释放机制

如图 5-5 所示，显式 KV 的难点不止是捕获状态，还包括证明下游确实继承了该状态，并在所有终态完成资源结算。StateBus 使用 engine-local handle 避免搬运大对象，以强身份与运行兼容签名限制消费范围，再通过调度接受证明和 Worker 前向证明确认真实使用。捕获本身会消耗资源，因此系统只在依赖连续、兼容条件确定且下游能够直接续接时启用；其他情况回到 Prefix Cache 或完整 Prefill。

第 6 章 总结

StateBus 已从概念设计发展为可运行的多智能体基础设施原型。系统实现规划者、检索者、执行者、总结者四角色协作，以 UDS 和 Protobuf 建立可验证控制合同，以 SemanticStateRef 和 LogitStateRef 传递可审计的非文本数值状态，以兼容门约束共享记忆复用，并通过基于提示词布局的前缀复用减少同一 vLLM 实例内的重复预填充计算。受限 Python CodeAct、结构化 DSL、独立工作区和提交质量门共同形成可信执行闭环。

在固定代码、模型和任务配置下，共完成 95/95 次正式执行。在质量保持的 10 个同任务对照中，总 Token 降低 47.40%、线路传输字节降低 64.85%、总耗时降低 6.32%；通信专项在相同消息数下降低控制面字节 83.05%；非文本状态 9/9 跨 PID 消费并改变选择；共享记忆实际使用为 35%，同时拒绝 81.25% 的不兼容候选；五类 25 个能力任务全部通过，18 个受限 Python CodeAct 和 7 个结构化 DSL 均未触发回退。这些结果共同说明：StateBus 的优势不只在少传文本，而在把协作升级为低通信负担、可验证、可追踪、可安全复用的状态系统。

两项补充受控诊断与上述 95 次正式执行分开统计。前缀复用专项在 40 个请求中保持两组 20/20 请求和输出合同通过，请求区间 Block 命中率由 0% 提高到 78.0160%，TTFT 和总延迟分别降低 88.3% 和 54.6%；LogitState 受控挑战将验证器通过结果由 8/12 提高到 12/12，将不可判定任务的错误工作进程放行由 2 次降为 0 次，并实现 19/19 个概率状态跨 PID 消费和释放。这些结果分别验证了重复预填充复用和数值状态改变控制流，不用于扩大正式任务分母或宣称额外 Token 收益。

StateBus Studio 进一步把工程机制转化为可操作产品。固定证据页保证本文、实验结果表与系统界面的数字一致；实时任务页展示类型化对象、智能体输入输出、模型生成程序、验证回执和最终结论；白名单 API、单工作进程队列与既有模型服务复用保证网页不会演变为任意命令执行入口。

后续工作包括：扩展更多行业任务与公开数据集；在更多模型、硬件、并发负载和缓存隔离条件下重复前缀复用实验；扩大 LogitState 阈值与候选别名偏差的校准规模，并评估减少额外模型调用的策略；增强长期记忆的压缩、失效和冲突解决；完善 Ref 租约与跨节点数据面；补充 openEuler 上的一键安装、离线镜像与长期稳定性验证。所有未来能力在完成代码和实验前均不并入当前正式结论。

参考文献

- [1] CHHIKARA P, KHANT D, ARYAN S, et al. Mem0: Building production-ready AI agents with scalable long-term memory[EB/OL]. 2025[2026-07-30]. <https://arxiv.org/abs/2504.19413>.
- [2] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems. 2020.
- [3] WU Q, BANSAL G, ZHANG J, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation[EB/OL]. 2023[2026-07-30]. <https://arxiv.org/abs/2308.08155>.
- [4] Google. Protocol Buffers Documentation[EB/OL]. [2026-07-29]. <https://protobuf.dev/>.
- [5] SQLite Consortium. SQLite Documentation[EB/OL]. [2026-07-29]. <https://www.sqlite.org/docs.html>.
- [6] JOHNSON J, DOUZE M, JÉGOU H. Billion-scale similarity search with GPUs[J]. IEEE Transactions on Big Data, 2019, 7(3): 535-547.
- [7] vLLM Project. Automatic Prefix Caching[EB/OL]. [2026-07-30]. https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/.
- [8] WANG X, CHEN Y, YUAN L, et al. Executable code actions elicit better LLM agents[EB/OL]. 2024[2026-07-29]. <https://arxiv.org/abs/2402.01030>.
- [9] containers. bubblewrap: Low-level unprivileged sandboxing tool[EB/OL]. [2026-07-30]. <https://github.com/containers/bubblewrap>.
- [10] openEuler Community. openEuler 24.03 LTS-SP3 Documentation[EB/OL]. [2026-07-29]. <https://docs.openeuler.org/>.

附录

附录 A：主要交付文件

文件	用途
docs/reports/项目说明书-总.docx	本项目说明书正式文件
docs/reports/项目说明书-总-正文.md	可追踪的说明书正文源
docs/reports/StateBus--项目流程图-20260727.drawio.xml	diagrams.net/draw.io 可编辑流程图，多页面 XML
docs/reports/项目说明书素材-20260727/	项目图、实验图和 Studio 截图
tools/generate_statebus_manual_assets.py	使用本地 Python 环境重新生成图表与 XML

Studio 主要接口

接口	作用
GET /api/v1/system/health	检查 API、工作进程、Python、Embedding 和既有 vLLM
GET /api/v1/catalog	返回任务、数据集和允许执行的 recipe
GET /api/v1/evidence/current	返回固定证据快照
GET/POST /api/v1/runs	查询历史或创建白名单运行
GET /api/v1/runs/{id}/events	提供 SSE 结构化事件流
GET /api/v1/runs/{id}/task-flow	返回智能体输入、转换、输出、代码和验证视图
GET /api/v1/runs/{id}/artifacts	返回产物、StateRef、MemoryRef 和回执索引
POST /api/v1/runs/{id}/cancel	取消排队或运行中的指定任务

附录 B：实验指标口径说明

总 Token：统计同一任务链中各角色模型调用的输入与输出 Token 总量。

线路传输字节：统计智能体通信线路上的实际传输字节，包括控制帧及其携带的数据。

控制面字节：仅统计动作、参数、引用、能力、回执和运行状态等控制面字段，不包含被引用对象的实体内容。

实际使用：记忆候选通过兼容校验、完成角色重绑定，并在当前任务中被真实消费的比例。

质量通过：任务输出同时满足任务合同、业务验证器、来源定位和引用要求。

附录 C：正式实验任务与数据集说明

C.1 正式实验构成

正式实验由 40 次同任务配置对照、20 次连续任务、10 个机制真实性用例和 25 个能力覆盖任务构成，共 95 次正式执行。同任务对照用于比较 Token、通信载荷、执行时间和质量；连续任务用于验证跨轮依赖、记忆兼容和实际使用；机制真实性用例用于检查协议传递、跨进程消费和行为效果；能力覆盖任务用于验证不同数据形态与分析操作。

C.2 两组连续任务

连续任务包含 Operating 与 Financial 两条各 10 轮的顺序任务链。后续轮次通过前序依赖和复用合同消费已经验证的事实、策略或执行产物，因此这 20 轮任务是两条连续工作流，而不是 20 个相互独立的问题。

运营连续任务链（10 轮）

轮次	当前输入	任务内容	前序依赖	主要验证目标
R1	疾病统计 CSV	建立 schema 并计算两列缺失率	无	缺失率 36.45%、38.79%；schema 产物存在
R2	疾病统计 CSV	计算平均病例数并定位死亡数极值	R1	均值 2,081,990；Nigeria；2010
R3	疾病统计 CSV	按阈值 3 执行 IQR 异常处理并比较均值	R1、R2	处理前 10,149.43；处理后 5,949.08
R4	天气 CSV	建立天气 schema 并计算平均风速	R1、R2	平均风速 5.979；schema 产物存在
R5	前四轮产物	汇总 schema、统计、异常产物与来源链	R1—R4	产物不少于 5 个，策略不少于 2 个
R6	天气 CSV	按月份计算平均风速	R4	1 月 7.17；12 月 5.52；分组产物存在
R7	天气 CSV	按阈值 3 统计气压异常点	R3、R4	异常点 111 个；异常产物存在

轮次	当前输入	任务内容	前序依赖	主要验证目标
R8	字段漂移 天气 CSV	解析字段别名并重新计算 平均风速	R4、R6	平均风速 7.500；保留别名 解析记录
R9	天气 CSV	替换风速异常并填充温度 缺失值	R4、R7	风速 5.76；温度 52.47；清 洗产物存在
R10	前九轮产 物	汇总统计、清洗、拒绝记 录与完整来源链	R1—R9	产物不少于 8 个，策略不少 于 3 个

财务连续任务链（10 轮）

轮次	当前输入	任务内容	前序依赖	主要验证目标
R1	标准跨期财报	抽取 ACME 2026Q1 营收	无	营收 120
R2	标准跨期财报	抽取 ACME 2025Q4 营收	R1	营收 109
R3	前两轮事实	计算 ACME 两季度营收差值	R1、R2	绝对差 11
R4	标准跨期财报	抽取 BETA 2026Q1 营收	R1	营收 87
R5	前序公司事实	比较 ACME 与 BETA 同期营 收	R1、R4	差额 33
R6	标准跨期财报	抽取 BETA 2025Q4 营收	R1、R4	营收 79
R7	前序季度事实	计算 BETA 两季度营收差值	R4、R6	绝对差 8
R8	字段漂移财报	解析字段别名并抽取 BETA 2025Q3 营收	R4、R6	营收 72；保留来源 定位
R9	标准跨期财报	拒绝不兼容候选后抽取 ACME 2025Q3 营收	R8	营收 98；保留兼容 拒绝记录
R10	前九轮事实	生成两家公司三季度趋势并 汇总来源链	R1—R9	两家公司趋势均为 increasing

C.3 五类能力覆盖任务

能力覆盖实验包含五类共 25 个独立任务，任务之间不编码跨轮依赖。正式运行中观察到 18 次受限 Python CodeAct 和 7 次结构化 DSL；该比例是运行时的实际路由结果，不是任务注册表预先指定的任务类别。

任务类别	数量	输入形态	任务内容	验证重点
财报指标抽取	8	离线财务语料	按公司、季度和指标定位营收、毛利率或营业利润	指标值与来源文档精确匹配
多期趋势分析	5	跨期财报 Markdown	计算季度序列、趋势方向、首尾差值和百分比	趋势方向及关键差值精确匹配
跨表关联分析	5	跨期财报 Markdown	按公司和季度对齐数据，计算差额或并行趋势	对齐值、差额和趋势满足 Gold
条件聚合	4	疾病与天气 CSV	计算缺失率、均值、极值和按月聚合	数值满足容差，规定产物存在
异常检测与清洗	3	疾病与天气 CSV	按 1.5 倍 IQR 检测异常、替换异常并填充缺失	异常数、清洗均值和清洗产物满足要求

C.4 数据来源与验证规则

实验数据由四类本地输入组成：包含 856 行数据的疾病统计 CSV、包含 8,736 行逐小时记录的天气 CSV、包含两家公司三个季度指标的紧凑跨期财报 Markdown，以及由 5 份合成报告组成的离线财务语料。另设置小型 schema-drift 输入，用于检查字段别名解析和不兼容历史拒绝，不将其描述为独立的大规模数据集。

正式判定同时覆盖任务合同、产物 schema、确定性事实和来源生命周期。文本或枚举字段采用 exact 检查，统计结果采用明确的数值容差；要求结构化产物的任务还必须提交可解析的 ArtifactRef、来源定位或消费引用。最终文本看起来合理并不足以通过测试，只有业务结果、对象状态和来源关系同时闭合，任务才会被记为 PASS。

C.5 显式 KV 状态继承实验任务

为验证显式 KV 状态继承在完整多智能体主链中的实际效果，实验使用 Nova 和 Orion 两份离线运营报告。每份报告均包含三个季度的营收、毛利率、运营费用、客户

流失率和准时交付率，由公司与指标组合形成 10 个不同任务。每个任务分别在完整重算和 KV 状态继承条件下执行，构成 10 组配对结果。

指标	Nova: Q1/Q2/Q3	Orion: Q1/Q2/Q3	单位
营收	142 / 156 / 169	184 / 197 / 211	百万美元
毛利率	36.8 / 37.4 / 36.2	41.2 / 40.5 / 39.7	%
运营费用	44 / 47 / 53	57 / 61 / 66	百万美元
客户流失率	2.8 / 3.1 / 4.0	3.2 / 3.6 / 4.4	%
准时交付率	95.7 / 93.6 / 89.9	96.4 / 94.1 / 90.8	%

两种条件使用相同的任务合同、原始报告、4096 Token 共享前缀、角色逻辑、生成预算、温度参数和随机种子。完整重算条件下，总结者重新提交并计算完整共享前缀；KV 状态继承条件下，执行者捕获共享前缀对应的 KV 状态，总结者仅提交 KV 状态句柄和新增后缀输入。实验关闭 APC、语义裁剪、历史回放和多次执行尝试机制，保证两种条件的核心差异仅为 KV 状态的捕获与继承。

每个任务均检查三个季度的指标值、单位、字段完整性和结构化产物核心内容。KV 状态继承条件还需满足继承 Token 数为 4096，capture、load 和 release 记录完整，回退为 0，并在任务结束后释放对应 KV 状态。两种条件的生成文本无需逐 Token 完全相同，但必需事实和结构化产物核心必须保持一致。

附录 D：StateBus Docker 与模型服务使用说明

本附录用于说明 StateBus 应用容器、模型服务、运行时开关和 StateBus Studio 的启动方法。StateBus 应用运行在一个 openEuler 容器中，模型推理支持两条路径：一是调用外部 OpenAI 兼容接口，二是在宿主机启动 Qwen3-32B vLLM 服务。两种路径共用同一套 StateBus Runtime 与 Studio，只需通过配置文件选择模型路由。文中涉及的服务地址、监听端口、目录位置、GPU 编号、模型名称和前缀长度均为可配置项；表格中的示例值和命令中的尖括号占位符应根据实际部署环境替换。

采用本地 vLLM 时，应用容器使用宿主机网络，并通过配置文件中指定的服务地址访问宿主机上的模型服务。应用容器本身不申请 GPU，也不在容器内启动 vLLM。模型服务使用的 GPU 设备由宿主机 vLLM 配置决定，启动前应确认所选设备具有足够显存且未被其他高显存任务占用。

说明：除特别说明外，以下宿主机命令均应在 **StateBus** 项目仓库根目录执行。首次部署前应确认 **Docker** 及 **Docker Compose** 可用，当前用户具有执行 **Docker** 命令的权限，并已准备 **Qwen3-32B** 模型目录或可用的外部模型接口。配置示例中的“<模型服务主机>”“<模型服务端口>”“<容器内项目目录>”等内容为占位符，不应原样写入正式配置。

D.1 Docker 配置与应用容器启动

首先复制本地 Compose 环境配置，并将容器内用户标识设置为当前宿主机用户的 UID 和 GID。这样，容器写入日志、运行目录和缓存目录时，生成文件仍归当前宿主机用户所有，可避免后续出现文件无法修改或删除的问题。

```
cp docker/.env.example docker/.env
sed -i "s/^STATEBUS_UID=.* /STATEBUS_UID=$(id -u)/" docker/.env
sed -i "s/^STATEBUS_GID=.* /STATEBUS_GID=$(id -g)/" docker/.env
```

docker/.env 中的主要配置项如下。首次使用建议保留默认值，先完成普通四角色链路的冒烟测试，再根据实验目标启用前缀复用、Logit Gate 或显式 KV 状态继承。

配置项	默认值/示例值	作用说明
STATEBUS_DOCKER_TARGET	embed	构建或选择包含 Runtime、Studio 以及本地语义嵌入依赖的镜像目标。
STATEBUS_EMBED_DEVICE	cpu	指定语义嵌入模型在 CPU 上运行，避免与宿主机 vLLM 争用 GPU 显存。
STATEBUS_LLM_CONFIG_FILE	由部署环境指定	选择模型路由配置文件；可指向本地 vLLM 配置，也可指向外部 API 配置，文件位置由项目在容器内的挂载目录决定。
STATEBUS_PREFIX_ALIGNMENT_MODE	independent	默认保持各角色提示布局独立；仅在前缀复用实验中切换为共同证据前缀布局。
STATEBUS_LOGIT_GATE_MODE	off	控制候选概率观测与受控重查；普通运行保持关闭。
STATEBUS_ENGINE_LOCAL_KV_MODE	off	控制显式 KV 状态继承实验；普通运行保持关闭。

随后在宿主机创建模型、缓存、日志、任务运行目录和容器用户目录。该步骤通常只需执行一次。若目录已经存在，`mkdir -p` 不会删除或覆盖其中的数据。

```
export STATEBUS_HOST_ROOT="${STATEBUS_HOST_ROOT:-$HOME/statebus}"
mkdir -p \
"$STATEBUS_HOST_ROOT/models" \
"$STATEBUS_HOST_ROOT/caches" \
"$STATEBUS_HOST_ROOT/logs" \
"$STATEBUS_HOST_ROOT/runs" \
"$STATEBUS_HOST_ROOT/work/container-home/.local"
```

Compose 配置默认使用镜像 `statebus-dev-openeuler:24.03-lts-sp3-embed`。若宿主机已经加载该镜像，可使用 `--no-build` 直接创建并启动容器，避免重新构建。第二条命令进入名为 `statebus-dev` 的应用容器。

```
docker compose --env-file docker/.env -f docker/compose.yaml up -d --no-build
docker compose --env-file docker/.env -f docker/compose.yaml exec statebus-dev bash
```

说明：若提示镜像不存在，应先按照项目交付说明构建或加载对应镜像；若提示端口或容器名称冲突，应先检查已有 Compose 实例。应用容器采用 `host` 网络，模型服务和 Studio 实际使用的监听端口应以部署配置为准，并确保未被其他进程占用。

进入容器后，需要先加载 StateBus 的 Python 路径、运行目录和模型路由环境，再执行冒烟测试。冒烟测试会验证容器环境、角色路径与模型配置是否能够被 Runtime 正确识别。

```
source /usr/local/bin/activate_statebus_container.sh
python -m statebus.runtime.smoke --role-path-mode local_vllm
```

说明：每次新开容器终端后都应重新执行 `activate_statebus_container.sh`。若当前选择的是外部 API 路由，仍可使用同一激活脚本，但应确保 `docker/.env` 中的 `STATEBUS_LLM_CONFIG_FILE` 已经指向外部 API 配置。

D.2 外部 API 模型路由

当宿主机不具备运行 Qwen3-32B 的条件，或希望接入已有模型服务时，可使用外部 OpenAI 兼容接口。首先复制模型路由配置模板和密钥环境变量模板，生成仅在本机使用的 `local` 文件。

```
cp deploy/statebus_llm.yaml.example deploy/statebus_llm.yaml.local
cp deploy/statebus_llm.env.example deploy/statebus_llm.env.local
```

在 `deploy/statebus_llm.yaml.local` 中填写服务商接口地址，并为规划、检索、执行和总结等角色配置实际可用的模型名称；在 `deploy/statebus_llm.env.local` 中写入 API 密钥。

随后将 `docker/.env` 中的模型配置路径设置为容器内可见的 `YAML` 文件位置。该位置取决于项目目录在容器内的挂载方式，不应绑定某一台机器的固定绝对路径。

```
STATEBUS_LLM_CONFIG_FILE=<容器内项目目录>/deploy/statebus_llm.yaml.local
```

说明：API 密钥文件不应提交到代码仓库，也不应复制到公开日志或截图中。建议限制 `deploy/statebus_llm.env.local` 的文件读取权限，并在更换服务商或密钥后重新创建应用容器，使新配置生效。

外部 API 路径不需要在宿主机启动本地 `vLLM` 进程，但 `StateBus` 无法控制远端推理引擎，因此不能开展自动前缀缓存命中率和显式 `KV` 状态继承测量。若启用 `Logit Gate`，外部接口必须支持按照请求返回候选 `Token` 的对数概率；不支持 `logprobs` 的服务只能运行普通角色链路。

D.3 本地 Qwen3-32B vLLM 模型服务

本地 `Qwen3-32B` 模型服务由宿主机上的独立 `vLLM` 运行环境提供。首先复制 `vLLM` 环境变量模板，再打印最终解析后的配置，重点检查模型目录、服务名称、监听地址与端口、GPU 编号以及宿主机 `vLLM` 运行环境是否符合当前机器的实际部署情况。上述参数均应通过本地配置文件维护。

```
cp deploy/vllm.env.example deploy/vllm.env.local
scripts/vllm/manage_qwen3_32b.sh print-config
```

`print-config` 会输出当前宿主机解析后的实际配置。为避免使用说明与某一台机器的目录结构、网络端口和硬件编号绑定，本文仅给出配置项含义与示例写法，实际值以 `deploy/vllm.env.local` 及启动日志为准。

配置项	示例配置	说明
model	<宿主机模型目录>	模型文件在宿主机上的实际存放位置，由 <code>deploy/vllm.env.local</code> 配置。
name	qwen3-32b（示例）	<code>OpenAI</code> 兼容接口对外提供的模型名称，可根据实际服务调整，应与 <code>StateBus</code> 模型路由配置保持一致。

配置项	示例配置	说明
endpoint	http://<模型服务主机>:<模型服务端口>/v1	StateBus 访问宿主机 vLLM 服务的接口地址。监听地址与端口均可配置；使用 host 网络时，应填写容器可达的宿主机地址。
GPU	<GPU 编号>	模型服务使用的 GPU 设备编号，由宿主机 vLLM 配置决定。
environment	<宿主机 vLLM 运行环境>	宿主机上用于启动 vLLM 的 Python 或 Conda 运行环境，不属于 StateBus 应用容器。

说明：model、endpoint、GPU 和 environment 均应根据宿主机实际情况在 deploy/vllm.env.local 中配置。其中，environment 表示宿主机上用于启动 vLLM 的运行环境，而不是 StateBus 应用容器内部环境。执行 print-config 时，应以终端输出的实际配置为准。

确认配置无误后，可通过管理脚本启动标准 OpenAI 兼容服务，并分别查看运行状态、健康检查结果和日志。首次启动需要加载模型，健康检查在模型加载完成前可能暂时失败，应结合日志判断服务是否仍在正常初始化。

```
scripts/vllm/manage_qwen3_32b.sh start
scripts/vllm/manage_qwen3_32b.sh status
scripts/vllm/manage_qwen3_32b.sh health
scripts/vllm/manage_qwen3_32b.sh logs
```

停止服务时，管理脚本只会终止由当前仓库 PID 文件记录的进程，不会主动结束其他用户或其他项目启动的 vLLM 服务。

```
scripts/vllm/manage_qwen3_32b.sh stop
```

scripts/vllm/start_qwen3_32b.sh 是前台启动入口，manage_qwen3_32b.sh 在其外层增加 PID 文件、日志文件和健康检查。若配置的服务端口上已经存在健康服务，但该服务并非由当前仓库启动，管理脚本会拒绝接管，以避免误停或覆盖其他模型服务。

说明：项目提供的标准服务示例启用自动前缀缓存，并配置候选 Token 对数概率的返回上限。该上限可按推理服务能力和实验需求调整。因此，同一个标准 vLLM 服务可同时支持普通角色路径、基于共同证据布局的前缀复用观测，以及 Logit Retry Gate。

D.4 Runtime 运行模式与开关

普通 StateBus 任务应先保持全部实验开关关闭。该配置用于验证四角色链路、结构化通信、状态引用、共享记忆和受控执行的基本功能，不引入前缀布局或显式 KV 状态继承的额外变量。

```
STATEBUS_PREFIX_ALIGNMENT_MODE=independent
STATEBUS_PREFIX_POLICY=off
STATEBUS_LOGIT_GATE_MODE=off
STATEBUS_ENGINE_LOCAL_KV_MODE=off
```

需要观察共享证据前缀是否触发 vLLM 自动前缀缓存时，在 docker/.env 中启用共同证据前缀布局和观测策略。observe 只记录前缀复用相关遥测，不改变任务质量门。

```
STATEBUS_PREFIX_ALIGNMENT_MODE=shared_evidence_prefix
STATEBUS_PREFIX_POLICY=observe
```

Logit Gate 可以与普通链路或前缀复用独立组合。telemetry 模式只记录候选概率状态及判定结果；retry_once 模式在门控条件不满足时允许一次受控重查，不会无限重试。

```
STATEBUS_LOGIT_GATE_MODE=telemetry
STATEBUS_LOGIT_GATE_MODE=retry_once
```

修改 docker/.env 后，需要重新创建应用容器，才能保证 Compose 环境变量被完整刷新。该命令不会重新构建镜像，也不会停止宿主机上的 vLLM 服务。

```
docker compose --env-file docker/.env -f docker/compose.yaml up -d --no-build --force-recreate
```

不同运行模式的推荐组合如下。进行机制对照时，应一次只改变目标机制，避免自动前缀缓存、共同证据布局和显式 KV 状态继承同时生效，导致实验结果无法归因。

使用场景	模型服务	主要开关	适用说明
普通四角色任务	外部 API 或标准 vLLM	全部保持 off/independent	用于日常功能验证和普通任务运行。
前缀复用观测	标准 vLLM	shared_evidence_prefix + observe	验证共同证据前缀与自动前缀缓存，不与显式 KV 实验混用。
Logit Gate	支持 logprobs 的 API 或标准 vLLM	telemetry 或 retry_once	仅观测候选概率，或允许一次受控重查。
显式 KV 对照	KV continuation 专用 vLLM	continuation 或 full_replay	两种模式成对运行；保持前缀布局 independent。

D.5 显式 KV 状态继承服务

显式 KV 状态继承与标准服务使用同一组已配置的 GPU 资源和模型服务端点，但需要以专用模式启动 vLLM，不能与标准服务同时运行。专用启动器会关闭自动前缀缓

存，确保测得的复用收益来自显式的 produce、continue 和 release 链路，而不是由 vLLM 自动命中共同前缀。

首次使用前，应在宿主机工作目录中生成私有 Bearer Token。工作目录可通过环境变量指定，不要求使用固定路径。Token 用于限制显式 KV 接口的访问权限，文件权限设置为 600 后，仅文件所有者可以读写。

```
export STATEBUS_WORK_ROOT="${STATEBUS_WORK_ROOT:-$HOME/statebus/work}"
mkdir -p "$STATEBUS_WORK_ROOT/vllm-qwen3-32b"
umask 077
openssl rand -hex 32 > "$STATEBUS_WORK_ROOT/vllm-qwen3-32b/kv_api.token"
chmod 600 "$STATEBUS_WORK_ROOT/vllm-qwen3-32b/kv_api.token"
```

随后在 `deploy/vllm.env.local` 中设置服务模式、Token 文件位置和 KV 引擎代际标识。Token 路径和代际标识均为可配置项。代际标识用于区分不同模型服务实例产生的 KV 状态，模型、引擎配置或服务代际变化后，不应继续复用旧代际的 KV 句柄。

```
export STATEBUS_VLLM_SERVICE_MODE="kv"
export STATEBUS_KV_API_TOKEN_FILE="${STATEBUS_WORK_ROOT}/vllm-qwen3-32b/kv_api.token"
export STATEBUS_KV_ENGINE_GENERATION="${STATEBUS_KV_ENGINE_GENERATION:-qwen3-32b-kv-generation-001}"
```

由于标准模式和 KV 模式使用同一配置端点，切换前必须先停止标准服务，再使用同一管理脚本启动 KV 模式。启动后应继续使用 `status`、`health` 和 `logs` 命令确认专用服务已完成模型加载。

```
scripts/vllm/manage_qwen3_32b.sh stop
scripts/vllm/manage_qwen3_32b.sh start
```

在 `docker/.env` 中启用 `continuation` 路径，并填写容器可见的 Token 路径、模型服务地址、模型名称和共享父前缀长度。应用容器使用 `host` 网络时，可采用宿主机可达地址；若监听地址、端口或目录挂载方式发生变化，应同步更新对应配置。Token 文件需要通过 StateBus 工作目录挂载到容器内。

```
STATEBUS_ENGINE_LOCAL_KV_MODE=continuation
STATEBUS_KV_API_BASE_URL=http://<模型服务主机>:<模型服务端口>
STATEBUS_KV_API_TOKEN_FILE=<容器内工作目录>/vllm-qwen3-32b/kv_api.token
STATEBUS_ENGINE_LOCAL_KV_MODEL=<模型服务名称>
STATEBUS_ENGINE_LOCAL_KV_PARENT_TOKENS=<共享父前缀 Token 数>
```

对照实验中，将 `STATEBUS_ENGINE_LOCAL_KV_MODE` 改为 `full_replay` 即可运行不使用显式 KV 继承的匹配基线。`full_replay` 与 `continuation` 应使用相同任务、共享前

缀长度、生成参数和验证规则。共享父前缀长度为实验参数，应结合模型上下文容量和任务设计配置，不应视为固定值。实验期间前缀布局应保持 independent，避免把自动前缀复用与显式 KV 继承混合。

说明：显式 KV 句柄只在当前引擎代际和相同模型配置下有效。任务结束后应确认 release 记录完整；若服务重启、模型更换或引擎代际变化，应将旧句柄视为失效对象，不得继续加载。

D.6 StateBus Studio 启动与访问

StateBus Studio 应从已经激活环境的应用容器终端中启动。脚本会启动 FastAPI 后端和前端静态服务，并使用 docker/.env 选定的外部 API 或本地 vLLM 模型路由。Studio 监听地址和端口由启动配置决定。

```
source /usr/local/bin/activate_statebus_container.sh
scripts/run_statebus_studio.sh
```

Studio 启动后，应根据启动日志中显示的监听地址和端口访问系统，例如 `http://<Studio 主机>:<Studio 端口>`。应用容器使用 host 网络时，宿主机浏览器可访问宿主机可达的监听地址。Studio 中的语义嵌入默认使用 CPU，不占用宿主机 vLLM 配置的 GPU 资源。证据中心读取固定证据快照，实时演示页面则通过白名单任务配方、单 Worker 队列和 SSE 事件流展示当前任务执行过程。

说明：若页面无法访问，应依次检查 Studio 进程是否仍在运行、配置的监听端口是否被占用、应用容器是否已正确重建，以及浏览器访问地址是否与启动日志一致。若服务仅绑定回环地址，则只能从本机访问；若需要从其他主机访问，应调整监听地址，并同步配置防火墙和访问控制。Studio 不负责启动或重启 vLLM，运行实时任务前应单独确认所选模型路由健康。

D.7 推荐启动顺序与常见检查

为减少配置错误，建议按照“确认模型路由—启动模型服务—启动应用容器—执行冒烟测试—启动 Studio”的顺序操作。使用外部 API 时可跳过本地 vLLM 启动步骤，但必须先验证接口地址、模型名称和 API 密钥；使用本地 vLLM 时，应先通过 health 确认模型服务可用，再进入容器执行 Runtime 冒烟测试。

当任务无法调用模型时，优先检查 STATEBUS_LLM_CONFIG_FILE 是否指向正确文件、YAML 中的模型名称是否与服务端一致，以及配置的模型服务地址是否可达并通

过健康检查。出现 GPU 显存不足时，应检查宿主机所选 GPU 上的其他进程；应用容器不申请 GPU，停止容器通常不能释放 vLLM 占用的显存。

当配置修改后表现未变化时，应区分配置所在位置：`docker/.env` 的修改需要强制重新创建应用容器；`deploy/vllm.env.local` 的修改需要停止并重新启动 vLLM 服务；YAML 路由和密钥文件修改后，建议重建容器并重新执行冒烟测试。切换标准 vLLM 与 KV 模式时，还应确认旧服务已经停止且配置的服务端口已释放。

日志目录、任务运行目录和工作目录均可通过部署配置指定，不要求绑定固定的宿主机路径。排查问题时应保留对应任务的 Run 目录、模型服务日志、Runtime 事件和验证报告；不要仅依据最终页面提示判断失败原因。涉及 API 密钥和 KV Token 的文件不得加入公开交付包或版本控制。